



VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

BRNO UNIVERSITY OF TECHNOLOGY

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

FACULTY OF INFORMATION TECHNOLOGY

ÚSTAV INTELIGENTNÍCH SYSTÉMŮ

DEPARTMENT OF INTELLIGENT SYSTEMS

**DEMONSTRACE VYUŽITÍ PROSTŘEDKŮ ROS 2 NA
PODVOZKU WAVE ROVER S RASPBERRY PI, KAME-
ROU A LIDAREM**

THESIS TITLE

BAKALÁŘSKÁ PRÁCE

BACHELOR'S THESIS

AUTOR PRÁCE

AUTHOR

FILIP BOTLO

VEDOUCÍ PRÁCE

SUPERVISOR

doc. Ing. VLADIMÍR JANOUŠEK, Ph.D.

BRNO 2024

Abstrakt

Táto práca sa zaoberá mobilnou robotikou a teleprezenciou. Cieľom je navrhnuť a implementovať systém riadenia robota na platforme WAVE ROVER s využitím Robot Operating System 2 (ROS 2). Práca zahŕňa prehľad metód vývoja softvéru pre mobilné roboty v ROS 2 a oboznámenie sa s platformou WAVE ROVER, overenie použiteľnosti existujúceho hardvéru a jeho rozšírenie o senzory (lidar, kamera). Navrhovaný systém umožňuje prepínanie medzi diaľkovým ovládaním a autonómnym pohybom s vyhýbaním sa prekážkam. Riadiaci počítač je Raspberry Pi 5 s Ubuntu 24.04.

Abstract

This thesis deals with mobile robotics and telepresence. The aim is to design and implement a robot control system on the WAVE ROVER platform using Robot Operating System 2 (ROS 2). The work surveys software development practices for mobile robots in ROS 2, introduces the WAVE ROVER platform, validates the existing hardware, and extends it with sensors (lidar, camera). The proposed system supports switching between remote control and autonomous motion with obstacle avoidance. The onboard computer is a Raspberry Pi 5 running Ubuntu 24.04.

Klíčové slová

Sem budou zapsána jednotlivá klíčová slova v českém (slovenském) jazyce, oddělená čárkami.

Keywords

Sem budou zapsána jednotlivá klíčová slova v anglickém jazyce, oddělená čárkami.

Citácia

BOTLO, Filip. *Demonstrace využití prostředků ROS 2 na podvozku WAVE ROVER s Raspberry Pi, kamerou a lidarem*. Brno, 2024. Bakalářská práce. Vysoké učení technické v Brně, Fakulta informačních technologií. Vedoucí práce doc. Ing. Vladimír Janoušek, Ph.D.

Demonstrace využití prostředků ROS 2 na podvozku WAVE ROVER s Raspberry Pi, kamerou a lidarem

Prehlásenie

Prehlasujem, že som túto bakalársku prácu vypracoval samostatne pod vedením doc. Janouška. Ďalšie informácie mi poskytol... Uviedol som všetky literárne pramene, publikácie a iné zdroje, z ktorých som čerpal.

.....

Filip Botlo
11. apríla 2026

Podakovanie

V této sekci je možno uvést poděkování vedoucímu práce a těm, kteří poskytli odbornou pomoc (externí zadavatel, konzultant apod.).

Obsah

1	Úvod	3
2	Hardware vybavenie	4
2.1	Raspberry Pi 5	4
2.2	General Driver Board for Robots	5
2.3	Wave Rover	6
2.4	LIDAR	9
2.5	Raspberry Pi Camera Module 3	10
2.6	Kamera na Raspberry Pi 5 s Ubuntu 24.04 a ROS 2 Jazzy	10
2.7	Komunikačné rozhrania	12
3	Robot Operating System 2	14
3.1	Úvod do ROS2	14
3.2	Práca s ROS2	15
3.3	Základné komponenty a nástroje ROS2	18
3.4	Integrácia senzorov a akčných členov	19
4	Implementácia	21
4.1	Postup implementácie v texte	21
4.2	Model robota, URDF/Xacro a simulácia v Gazebo a RViz	22
4.3	Aktuálna architektúra robota (autíčka)	25
4.4	Riadenie pohybu, optický tok, IMU PID a autonómne bludenie	29
4.5	Návrh uzlov systému pre ROS2	45
4.6	Implementácia uzlov a komunikácia s UGV Base General	47
5	Záver	51
	Literatúra	52

Zoznam obrázkov

2.1	Špecifikácie Raspberry Pi 5 (prehľad platformy)	4
2.2	General Driver Board for Robots	6
2.3	Wave Rover – tri pohľady	7
2.4	Princíp fungovania akcelerometra	8
2.5	Princíp fungovania gyroskopu	8
2.6	Komplementárny filter [1]	9
2.7	Meranie vzdialenosti LiDARom	9
2.8	Raspberry Pi Camera Module 3	10
2.9	Pripojenie zariadení na spoločnú I ² C zbernicu	12
2.10	Príklad prenosu dát po I ² C	13
4.1	Model robota v RViz2	24
4.2	Simulovaný model Wave Rovera v prostredí Gazebo Sim	25
4.3	Graf uzlov: <code>runtime_stack</code>	27
4.4	Webové teleop rozhranie	28
4.5	Priebeh IMU a korekcia pohonu	30
4.6	Optický tok: debug výstup	32
4.7	Optický tok: korekcia <code>angular.z</code>	33
4.8	Optický tok: <code>mean_dx</code>	33
4.9	Bludenie: laser v <code>rqt</code>	34
4.10	Bludenie: blokový diagram	34
4.11	Graf uzlov: manuál + SLAM	40
4.12	Sledovanie objektu: debug obraz	44
4.13	Sledovanie objektu: priebeh rýchlostí	45

Kapitola 1

Úvod

Práca sa zaoberá výzvami mobilnej robotiky a teleprezencie, návrhom a implementáciou riadiaceho systému pre mobilného robota. Základnou platformou pre túto prácu je WAVE ROVER vybavený lidarom D300 LD19. Ako hlavný softvérový nástroj sa uvádza rámec ROS2 vzhľadom na to, že sa stal štandardom pri riadení a koordinácii robotických systémov. Cieľom tejto práce je nielen ukázať, čo všetko sa dá pomocou ROS2 na zvolenej hardvérovej platforme urobiť, ale navrhnúť systém, ktorý sa prispôsobí pri konfigurácii a experimentovaní s alternatívnymi komponentmi.

S úlohou úzko súvisia dve príbuzné bakalárske práce z tej istej fakulty. Práca Skoleka [33] sa zaoberá vyvažovaným dvojkoľosovým robotom s ROS 2 na Raspberry Pi 4B: využíva PID regulátory na základe údajov z IMU a enkodérov, diaľkové ovládanie cez Bluetooth a autonómny pohyb s vyhýbaním sa prekážkam pomocou ultrazvukového senzora; dôležitá je modularita systému (balíčky pre senzory, riadenie, Bluetooth) a spúšťanie cez launch súbory.

Bakalárska práca Daniela Onderku *Mobilní kamera realizovaná prostředky ROS2* [17] (katalógový záznam FIT VUT) bola **orientačnou inšpiráciou** pri budovaní vlastného riešenia: kombinácia **teleprezencie** (kamera v ROS2), **prepínania medzi manuálnym a autonómnym správaním**, nasadenia **Gazebo** na overenie funkcionality v simulácii a integrácie **slam_toolbox** a **Nav2** pre mapovanie a globálnu navigáciu. Onderka pracoval s platformou Adept AWR 4WD a ROS2 na RPi4; táto práca prenáša podobnú myšlienku na WAVE ROVER, RPi5, LiDAR LD19 a vlastný webový teleop. Rozdiely: namiesto **ros2_control** je u fyzického podvozku primárne priame riadenie motorov (Motor Driver HAT alebo alternatívne HTTP na ESP32); dynamické prepínanie režimov je riešené parametrami uzlov a službami (sek. 4.4).

Obe referenčné práce demonštrujú možnosti ROS2 a zmysluplné rozdelenie uzlov; táto práca z nich čerpá najmä **architektonický vzor** (launch súbory, senzorové témy, rozšírenie o ďalší hardvér), nie kopírovanie konkrétneho kódu.

V práci sa skúmajú metódy vývoja softvéru pre mobilné roboty, zoznamuje sa s možnosťami a obmedzeniami hardvéru WAVE ROVER a následne sa analyzuje použiteľnosť súčasných senzorov a riadiacich jednotiek motorov. V prípade potreby sú zapracované návrhy a implementácie týkajúce sa zmien v hardvéri vrátane výmeny motorov za motory s enkodérom pre schopné presné riadenie pohybu robota.

Navrhovaný systém má možnosť prepnutia medzi diaľkovým a autonómnym ovládaním. Proces vývoja práce umožní dynamicky rekonfigurovať samotný riadiaci rámec, takže je možné experimentovať s rôznymi konfiguráciami hardvéru.

Kapitola 2

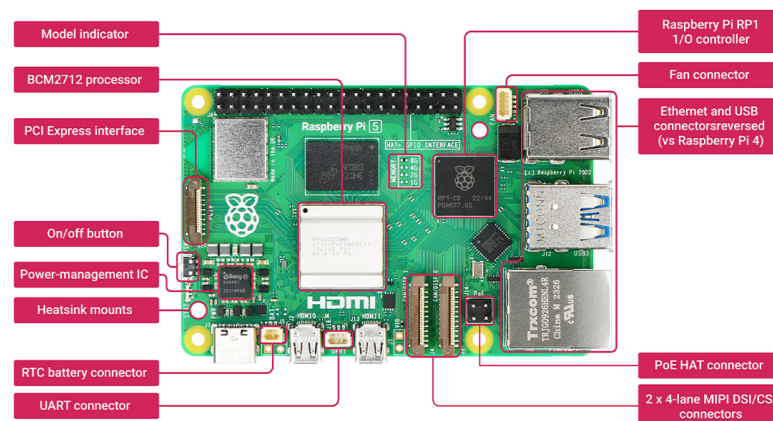
Hardware vybavenie

2.1 Raspberry Pi 5

V nasledujúcej sekcii poskytnem prehľad o Raspberry Pi 5, čo je najnovšia generácia týchto jednodoskových počítačov. Sekcia poukazuje na jeho hlavné vlastnosti a zmeny oproti predchádzajúcim modelom.

Raspberry Pi 5

Raspberry Pi 5 je kompaktný jednodoskový počítač určený na vývoj a prototypovanie. Ako cenovo dostupná, no výkonná platforma podporuje vzdelávanie a projekty v oblasti elektroniky, programovania, internetu vecí a robotiky [26].



Obr. 2.1: Špecifikácie Raspberry Pi 5 (prehľad platformy)

Ako funguje

Raspberry Pi 5 je založený na architektúre ARM a využíva procesor Cortex-A76[25], ktorý je vybavený štyrmi jadrami pracujúcimi na frekvencii až 2,4 GHz. Táto architektúra a taktová frekvencia umožňujú paralelné spracovanie viacerých úloh, čo je výhodné pri výpočtovo náročných úlohách, akými sú spracovanie senzorov a riadenie robota. Na platforme Wave Rover je Raspberry Pi pripojené cez GPIO piny [26], ktoré je možné použiť ako vstupy na čítanie signálov alebo ako výstupy na ovládanie.

Porovnanie s Raspberry Pi 4B

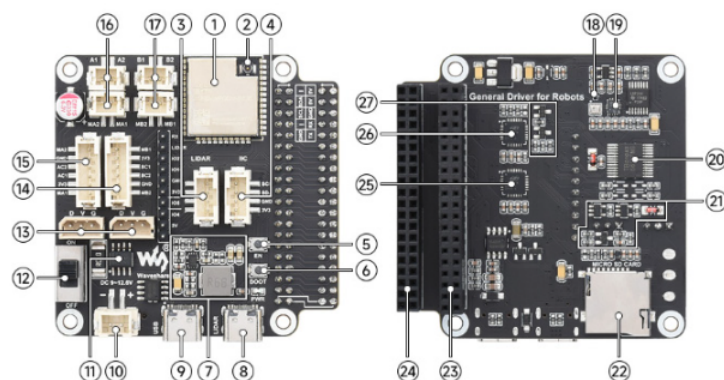
Prehľad hlavných rozdielov je v tabuľke 2.1; pre túto prácu je rozhodujúci najmä vyšší výpočtový výkon pri súbežnom behu ROS 2, kamery a navigácie.

Tabuľka 2.1: Stručné porovnanie Raspberry Pi 5 a Pi 4B (vybrané body)

Oblasť	Pi 4B	Pi 5
CPU	Cortex-A72	Cortex-A76 (vyšší výkon na jadro; vhodnejšie na ROS 2 + obraz)
PCIe	–	natívne PCIe (rozšírenia úložiska/periférií)
Grafika	jeden 4K výstup	dva 4K pri 60 Hz (menej kritické pre robota, užitočné pri vývoji)

2.2 General Driver Board for Robots

Driver Board slúži ako medzičlánok medzi mikrokontrolérom, napríklad Raspberry Pi, a motormi robota. Umožňuje bezpečné a efektívne ovládanie motorov, ktoré zahŕňa riadenie rýchlosti, smeru a celkového výkonu pohybu robota.



1. **ESP32-WROOM-32 controller module**
Can be developed using Arduino IDE
2. **IPEX1 WiFi connector**
For connecting WiFi antenna to increase the wireless communication distance
3. **LIDAR interface**
Integrated radar adapter function
4. **I2C peripheral expansion interface**
For connecting with OLED screen or other I2C sensors
5. **Reset Button**
Press and release to reboot the ESP32
6. **Download button**
The ESP32 will enter the download mode after powering on
7. **DC-DC 5V voltage regulator circuit**
Power supply for host computers such as Raspberry Pi or Jetson nano
8. **Type-C port (LADAR)**
LIDAR data transmission
9. **Type-C port (USB)**
ESP32 communication interface, for uploading programs to ESP32
10. **XH2.54 power port**
Support DC 7-13V input, can directly power the serial bus servos and motors
11. **INA219**
voltage/current monitoring chip
12. **Power ON/OFF**
External power supply ON/OFF
13. **ST3215 serial bus servo interface**
For connecting with ST3215 serial bus servo
14. **Motor interface PH2.0 6P**
Group B interface for motor with encoder
15. **Motor interface PH2.0 6P**
Group A interface for motor with encoder
16. **Motor interface PH2.0 2P**
Group A interface for motor without encoder
17. **Motor interface PH2.0 2P**
Group B interface for motor without encoder
18. **AK09918C**
3-axis electronic compass
19. **QM18658C**
6-axis motion sensor
20. **TB6612FNG**
Motor Control Chip
21. **Serial bus servo control circuit**
for controlling multiple ST3215 serial bus servos and obtaining servos feedback
22. **TF card slot**
Can be used to store logs or WiFi configurations
23. **40PIN GPIO header**
For connecting with Raspberry Pi or other host boards
24. **40PIN extended header**
Easy to use the GPIO pins of Raspberry Pi or other host boards
25. **CP-2102**
UART to USB, for radar data transmission
26. **CP-2102**
UART to USB, for ESP32 communication
27. **Automatic download circuit**
For Uploading programs to the ESP32 without pressing the EN and BOOT buttons

Obr. 2.2: General Driver Board for Robots

Ako funguje Driver Board

Driver Board prijíma signály z mikrokontroléra, ktoré obsahujú informácie o smere a rýchlosti pohybu. Tieto signály sú spracované tranzistormi alebo H-mostmi na doske ovládača, ktoré zabezpečujú riadenie prúdu pre motory. Ak napríklad mikrokontrolér vyšle signál PWM s rôznymi frekvenciami, doska ovládača tento signál prevedie na riadenie prúdu do motora, čím riadi jeho rýchlosť. Ak mikrokontrolér zmení polaritu signálu, potom doska ovládača zmení smer prúdu a motor sa otáča opačným smerom. Takto je možné plynule meniť rýchlosť a smer otáčania motorov [34].

2.3 Wave Rover

Wave Rover je experimentálna platforma pre mobilnú robotiku, ktorá umožňuje vývoj a testovanie širokej škály softvérových a hardvérových aplikácií. Tento robotický systém je navrhnutý tak, aby zvládol pohyb na rôznych povrchoch a poskytoval údaje na analýzu

pohybu a navigáciu. Wave Rover má viacero senzorov a aktuátorov, ktoré umožňujú interakciu s prostredím a poskytujú flexibilitu pri modifikácii a rozširovaní hardvéru podľa potrieb výskumu a vývoja.

Stavba tela

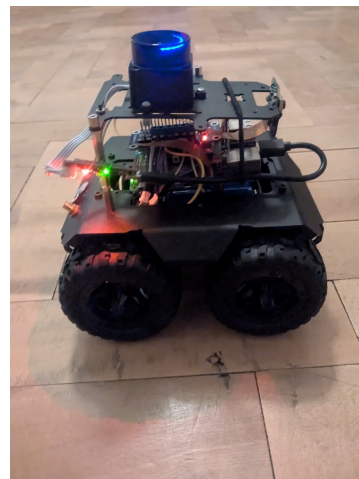
Konštrukcia kladie dôraz na stabilitu a odolnosť na rôznych povrchoch. Rám kombinuje dostatočnú pevnosť s nízkou hmotnosťou, chráni batérie, motory a senzory a uľahčuje vyváženie a ovládateľnosť podvozku [35].



Pohľad spredu



Šikmý (diagonálny) pohľad



Pohľad z boku

Obr. 2.3: Platforma Wave Rover v nasadenej zostave: pohľad spredu, šikmo zhora a z boku

Modularita a rozširiteľnosť

Telo robota je navrhnuté tak, aby bolo možné ľahko pripojiť a vymeniť komponenty. V ráme sú štandardizované montážne body pre ďalšie ultrazvukové senzory, LiDAR, kameru či ďalšie moduly. Rozšírenie umožňujú aj GPIO piny a zbernice I²C, SPI, UART a PWM.

Arduino Nano ako doplnková jednotka pre IMU

Okrem Raspberry Pi a Motor Driver HAT je v realizačnej zostave použité **Arduino Nano** (alebo ekvivalent s rovnakým rozhraním) ako malý mikrokontrolér výhradne na čítanie MPU-6050 a streamovanie údajov po USB. Tým sa senzor *nezdieľa* I²C zbernicu s motorovým HATom na strane RPi; ide o zámerné oddelenie kvôli spoľahlivosti (viď sek. 4.6). Nano je napájané z rovnakého 5 V logického prostredia ako bežné Arduino periférie; fyzické uchytienie zodpovedá montáži na podvozku (krátke I²C vedenie medzi modulom MPU-6050 a Nanom).

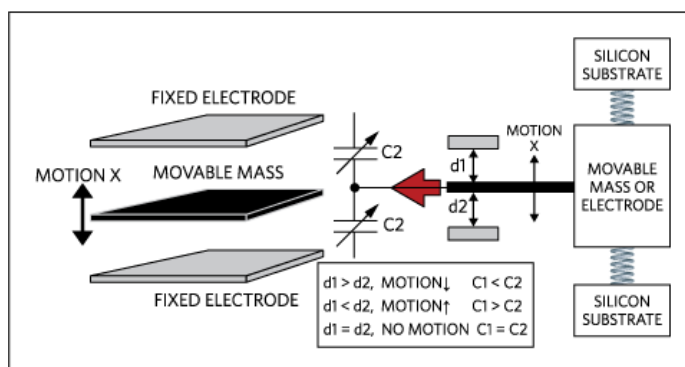
Inerciálny merací systém (IMU)

Wave Rover používa modul s čipom **MPU-6050**, ktorý integruje akcelerometer a gyroskop na jednom čipe; technické parametre rozsahov a rozhrania zodpovedajú dokumentácii čipu a komunite okolo knižníc pre MPU6050 [8]. V nasadenej zostave s **Motor Driver HAT** na Raspberry Pi nie je senzor pripojený priamo na I²C zbernicu počítača Raspberry Pi spolu

s ovládačom motorov (PCA9685) – namiesto toho slúži ako meracia hlava na samostatnom **Arduino Nano** s **vlastným firmwareom** a údaje prichádzajú na RPi cez USB sériový port (podrobnejšie odôvodnenie a popis firmwaru v sekcii 4.6). MPU-6050 poskytuje údaje o pohybe a orientácii potrebné pre korekciu driftu, otáčanie pri bludení a ďalšie vrstvy riadenia. Nižšie nasleduje stručný popis fyzikálneho princípu akcelerometra a gyroskopu a fúzie signálov.

Akcelerometer

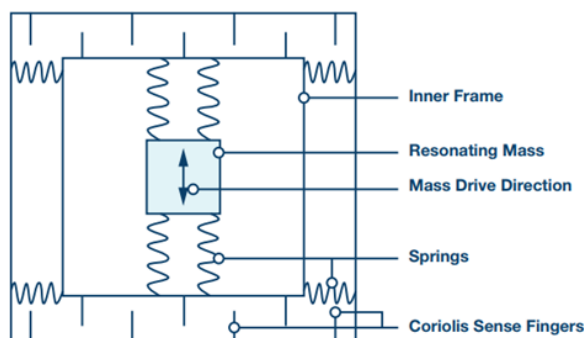
V module IMU MPU-6050 je trojosový akcelerometer, ktorý meria lineárne zrýchlenie. MEMS akcelerometer využíva väčšinou kapacitný princíp: výchylka hmotného prvku mení kapacitu štruktúry a tá sa premení na merateľný elektrický signál [2].



Obr. 2.4: Princíp fungovania akcelerometra

Gyroskop

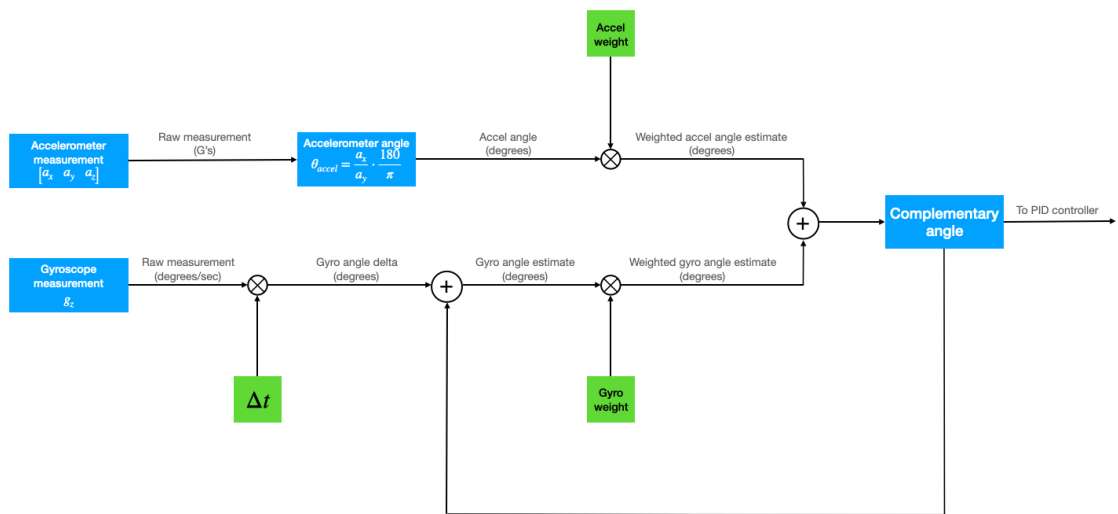
Gyroskop meria uhlovú rýchlosť; v MEMS realizácii sa často využíva Coriolisova sila pôsobiaca na vibrujúci prvok, pričom výsledkom je signál úmerný rýchlosti otáčania [9].



Obr. 2.5: Princíp fungovania gyroskopu

Fúzia údajov pre navigáciu

Údaje z akcelerometra a gyroskopu sa zvyčajne kombinujú (napríklad komplementárnym alebo Kalmanovým filtrom), aby sa získala stabilnejšia odhadnutá orientácia. V nasadenej zostave sa vo firmware pre Arduino Nano používa komplementárny filter (obr. 2.6).



Obr. 2.6: Komplementárny filter [1]

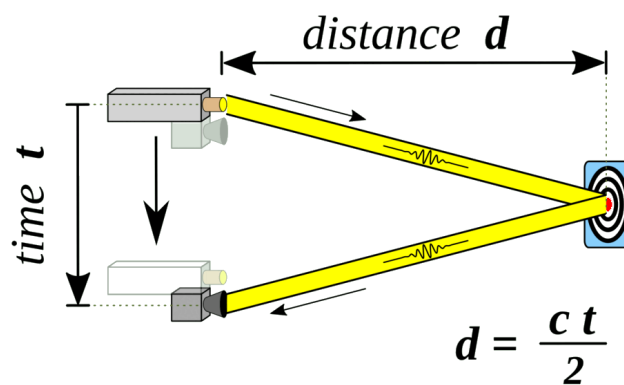
2.4 LIDAR

Čo je LIDAR

LIDAR (Light Detection and Ranging) je technológia, ktorá využíva laserové svetlo na meranie vzdialeností, ako aj na vytváranie trojrozmerných modelov prostredia.

Princíp fungovania LiDAR

Snímač vysiela krátke impulzy (alebo modulované kontinuálne svetlo) a z merania času návratu alebo fázového posunu odvodza vzdialenosť k odrazovej ploche. LiDAR sa bežne používa v mobilnej robotike, autonómnych vozidlách a geodézii [5].



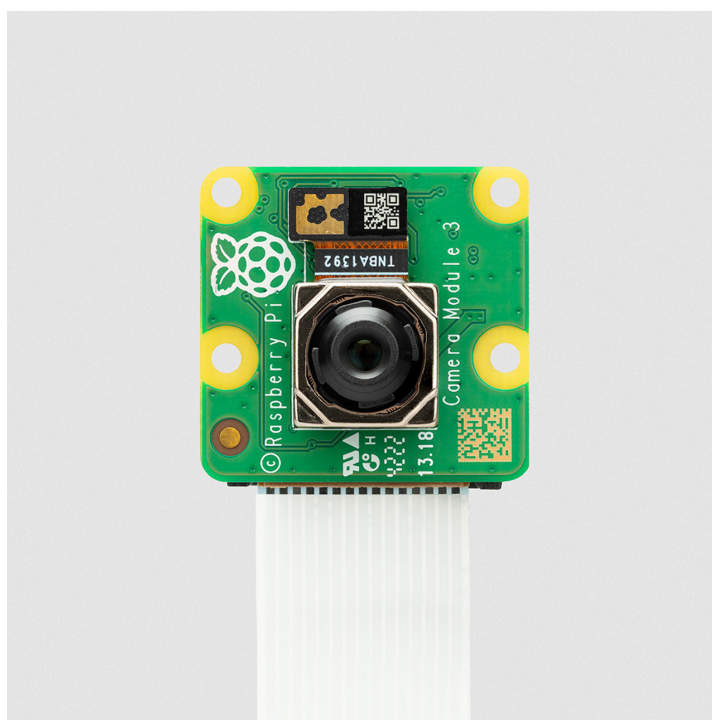
Obr. 2.7: Meranie vzdialenosti LiDARom

Rotujúce alebo viaclúčové snímače dokážu pokryť okolie v plnom azimute; merania sa ukladajú ako mračno bodov alebo ako 2D prierez (napr. **LaserScan** v ROS), čo umožňuje mapovanie a detekciu prekážok v autonómnych mobilných systémoch [5].

2.5 Raspberry Pi Camera Module 3

V tejto práci je na platforme Wave Rover použitý oficiálny modul **Raspberry Pi Camera Module 3** pripojený k Raspberry Pi 5 cez **CSI** rozhranie (flexibilný pásikový kábel). Modul využíva snímač typu **Sony IMX708** (približne 12 MP), podporuje **HDR** snímanie a **fázové zaostrovanie (PDAF)**, čo uľahčuje ostrenie pri rôznych vzdialenostiach objektov v scéne – praktické pri teleprezencii aj pri algoritmoch počítačového videnia [27].

Kamera v systéme slúži predovšetkým na **teleprezenciu** (stream do webového rozhrania), **vizualizáciu** v RViz a na experimenty s **optickým tokom** a **sledovaním farebného objektu**. Podrobný postup integrácie do ROS 2 (balík `camera_ros`, `libcamera`, konfigurácia na Ubuntu 24.04) je v nasledujúcej sekcii 2.6.



Obr. 2.8: Raspberry Pi Camera Module 3 (Sony IMX708) pripojená k Raspberry Pi 5 cez CSI pásikový kábel; modul je uchytený na podvozku Wave Rover

2.6 Kamera na Raspberry Pi 5 s Ubuntu 24.04 a ROS 2 Jazzy

Integrácia kamery na Raspberry Pi 5 s operačným systémom Ubuntu 24.04 (Noble) a ROS 2 Jazzy vyžaduje špecifický postup oproti starším verziám OS alebo iným kamerám. Táto sekcia stručne popisuje odôvodnenie zvoleného riešenia a používané komponenty podľa dokumentácie inštalácie pre RPi5 [3].

Prečo nie štandardný v4l2 ovládač

Na Raspberry Pi 5 s Ubuntu 24.04 nie je vhodné používať balík `ros-jazzy-v4l2-camera`. Kameraný subsystém RPi5 využíva *libcamera* (resp. nástupcu *rpicam*), nie klasický V4L2 legacy stack. Preto sa používa balík `camera_ros` [28], ktorý pracuje s *libcamera* a publikuje obraz na ROS 2 témy.

Inštalácia a konfigurácia (prehľad)

Typický postup zahŕňa:

1. pridanie používateľa do skupiny `video`;
2. inštaláciu knižníc *libcamera* z PPA (napr. Marco Sonic rasppios) pre arm64 a Ubuntu 24.04;
3. úpravu konfiguračného súboru firmware (napr. `/boot/firmware/config.txt`): `camera_auto_detect=0` a `dtoverlay=imx708,cam0` pre Pi Camera 3;
4. *udev* pravidlo pre DMA heap (napr. zariadenie `dma_heap` so skupinou `video` a právami zápisu pre skupinu);
5. inštaláciu balíkov `ros-jazzy-camera-ros` a voliteľne `ros-jazzy-image-transport-plugins`.

Po reboote sa kamera spúšťa príkazom `ros2 run camera_ros camera_node` (prípadne s parametrami `width` a `height`).

Témy a uzly z camera_ros

Balík `camera_ros` publikuje napr. témy `/camera/image_raw`, `/camera/camera_info` a komprimovanú vetvu `/camera/image_raw/compressed`. Pre teleprezenciu alebo ďalšie spracovanie obrazu v ROS 2 sa môže použiť komprimovaný topic (`CompressedImage`) na zníženie záťaže siete; prípadne `image_transport republish` z `image_raw` na `compressed`.

Kalibrácia a camera_info

Ak nie je k dispozícii kalibračný súbor, uzol `camera_ros` môže hlásiť chýbajúci súbor v `~/.ros/camera_info/` (napr. pre `imx708_..._640x480.yaml`). Odporúča sa vykonať kalibráciu podľa návodu (napr. NAV2 camera calibration) a uložiť výsledný YAML s názvom zodpovedajúcim očakávanému `camera_name` a rozlíšeniu.

Vizualizácia (RViz2, rqt_image_view)

V ROS 2 sa na zobrazenie obrazu z kamery používa `rqt_image_view` s témou `/camera/image_raw` (alebo komprimovaný topic). RViz2 umožňuje pridať zobrazenie `Image` a zvoliť topic s obrazom. Pri vizualizácii ďalších senzorov (napr. LIDAR) v tej istej scéne sa nastaví spoločný `Fixed Frame` (napr. `base_link` alebo `base_laser`) a konzistentné `frame_id` v správach. Pri module Raspberry Pi Camera ide o CMOS senzor; konštrukciu snímania popisuje výrobca [27]. Farebné informácie sa získavajú cez masku farebných filtrov pred snímačom; interpoláciu z mozaiky (demosaiicing) podrobne rozoberajú napr. [10].

Súvis s implementáciou v tejto práci

Na reálnom robote sa obraz z CSI kamery získava uzlami balíka `camera_ros`, ktoré publikujú typicky témy `/camera/image_raw` a `/camera/camera_info`. Na zníženie záťaže CPU a siete sa často používa komprimovaný obraz (`sensor_msgs/msg/CompressedImage`) na témach s príponou `/compressed`. V projekte je voliteľný uzol `waverower_camera` (balík `waverower`) na ďalšie spracovanie; pre webové ovládanie sa obraz čerpá z komprimovaného topicu podľa konfigurácie rozhrania.

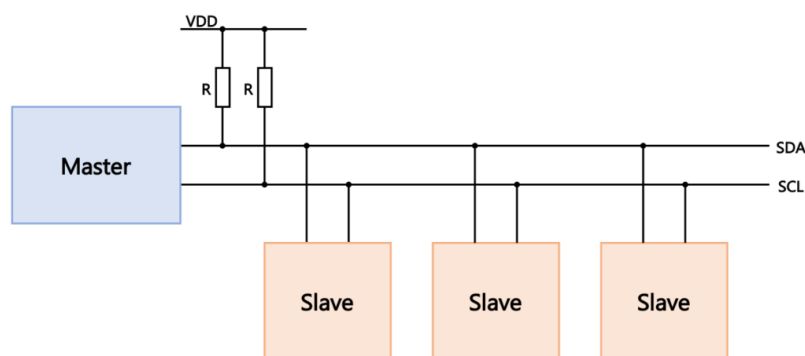
Korekcia dráhy z obrazu využíva knižnicu OpenCV podľa tutoriálu o optickom toku [24]. V launch súboroch (napr. `runtime_stack.launch.py`) beží uzol `optical_flow`: **riedky tok** Lucas–Kanade sleduje vybrané body medzi snímkami (`calcOpticalFlowPyrLK`). Voliteľne je k dispozícii aj **hustý tok** Farnebäck (`calcOpticalFlowFarneback`) ako samostatný spustiteľný súbor; v texte nižšie sa predpokladá Lucas–Kanade, ak nie je uvedené inak. Uzol z komprimovaného JPEG počíta stredný horizontálny posun bodov a z neho odvádza korekciu `angular.z`; tú pripočíta k poslednému `Twist` z teleopu a publikuje na `/teleop_cmd_vel_corrected`. Korekcia sa uplatní len pri jazde vpred a ak operátor aktívne netočí (`angular.z` v deadzone).

Na tvorbu mapy (SLAM) a na reaktívne autonómne správanie slúži predovšetkým LiDAR na téme `/scan` — na reálnom robote aj v simulácii. **Globálna navigácia Nav2** je však v rámci tejto práce zapojená a overená len v simulácii Gazebo, nie na fyzickom podvozku. IMU (`/imu`) môže byť použitá na doplnkovú stabilizáciu alebo odhad orientácie podľa konfigurácie. Kamera slúži predovšetkým teleprezencii, vizualizácii v RViz a experimentom s počítačovým videním; väzba na mapovanie je v tejto práci typicky cez LiDAR (nie priama vizuálna SLAM vetva).

2.7 Komunikačné rozhrania

I2C komunikačné rozhranie

I2C je sériové komunikačné rozhranie, ktoré umožňuje pripojenie rôznych zariadení na spoločnú dvojvodičovú zbernicu. Vďaka svojej jednoduchšej architektúre je vhodné pre nízko-rýchlostné periférne zariadenia, ako sú snímače, displeje alebo rozširujúce moduly.

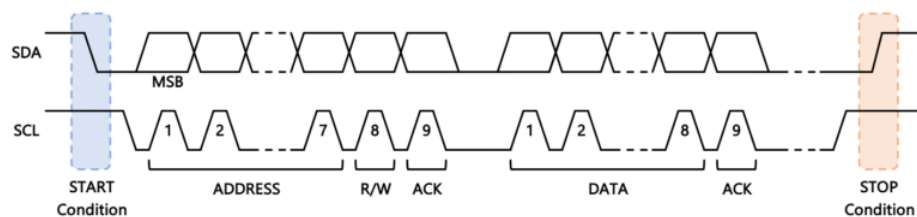


Obr. 2.9: Pripojenie zariadení na spoločnú I²C zbernicu

Základné vlastnosti:

- I²C používa iba dva vodiče: SDA (dátová linka) a SCL (hodinový signál), takže na jednu zbernicu je možné pripojiť viacero periférií.

- Komunikácia prebieha medzi jedným *masterom*, ktorý iniciuje prenos, a jedným alebo viacerými *slave* zariadeniami.
- **Jedinečné adresovanie:** Každé zariadenie má jedinečnú adresu, ktorú master použije na identifikáciu a komunikáciu s konkrétnym zariadením na zbernici.
- **Prenos dát:** Dáta sa prenášajú po bajtoch s potvrdením (ACK) po každom bajte. Komunikácia končí signálom STOP. [12]



Obr. 2.10: Príklad prenosu dát po I²C

Kapitola 3

Robot Operating System 2

3.1 Úvod do ROS2

Táto časť poskytuje prehľad o ROS2, jeho histórii, vývoji a vylepšeniach oproti predchádzajúcej verzii ROS1. Prejdeme si aj výhody, ktoré ROS2 prináša do moderných robotických aplikácií, a prečo sa považuje za najlepšie riešenie pre mobilnú robotiku a teleprezenciu.

Robot Operating System 2

ROS 2 (Robot Operating System 2) je moderná softvérová platforma tvorená z viacerých knižníc a nástrojov na vývoj v oblasti robotiky. Bola navrhnutá s cieľom prekonať obmedzenia prvej verzie a reagovať na požiadavky súčasných robotických systémov.

ROS 2 umožňuje efektívnu komunikáciu medzi týmito komponentmi vďaka použitiu štandardu DDS (Data Distribution Service), ktorý je osvedčený aj v kritických oblastiach, ako sú vojenské či finančné systémy. DDS prináša aj výhody, ako sú zabudovaná bezpečnosť, podpora pre reálne časové aplikácie a schopnosť pracovať v nestabilných sieťových podmienkach, napríklad pri Wi-Fi alebo satelitnej komunikácii.^[11] Štandard DDS a jeho úloha v dátocentrickom modeli sú zhrnuté v materiáloch DDS Foundation ^[16].

Rozdiel medzi ROS 1 a ROS 2

Jednou z najvýznamnejších zmien je odstránenie centrálného uzla *ROS master* z ROS 1: ak master prestal odpovedať, komunikácia medzi uzlami prestala fungovať. ROS 2 využíva decentralizovaný model založený na DDS; uzly sa objavujú v spoločnej doméne a nadväzujú spojenia podľa dokumentácie ^[19].

ROS 2 umožňuje nastavovať kvalitu služby (Quality of Service, QoS) pri publikovaní a odberu správ, čo pomáha pri synchronizácii, stratách paketov a požiadavkách na odozvu ^[20].

ROS 2 podporuje aj zostavovanie viacerých uzlov do jedného procesu ako *komponenty*, čo znižuje režijné náklady komunikácie.

ROS 2 oficiálne podporuje viac platforiem než ROS 1 (okrem Linuxu aj Windows a macOS), čo uľahčuje vývoj a integráciu nástrojov na rôznych počítačoch ^[11].

3.2 Práca s ROS2

Táto sekcia stručne predstavuje základné prvky ROS 2: uzly (*nodes*), témy (*topics*), služby (*services*) a akcie (*actions*).

Uzly

Uzly sú základnými stavebnými prvkami systému ROS2, pričom každý uzol predstavuje samostatný proces vykonávajúci špecifickú úlohu, napríklad spracovanie údajov zo senzorov alebo výpočty pre navigáciu. Uzly medzi sebou komunikujú pomocou tém, služieb alebo akcií. Model témy, publisher a subscriber a súvis so správami je podrobne rozvedený v tutoriáli ROS 2 [23]; štandardné typy správ sú definované v repozitári *common_interfaces* [21].

Metóda `spin()` v uzle spracováva prichádzajúce správy a udržiava uzol aktívny v slučke udalostí.

Vytvorenie uzla: V ROS2 sa uzol vytvára pomocou triedy `Node` v rámci knižnice `rclpy`. Uzol musí byť inicializovaný a registrovaný v rámci ROS2 siete. [22]

Algorithm 1 Implementácia jednoduchého uzlu

```
1: class TurtleBot4FirstNode(Node):
2:     def __init__(self):
3:         super().__init__('turtlebot4_first_python_node')
4:     def main(args = None):
5:         rclpy.init(args)
6:         node = TurtleBot4FirstNode()
7:         rclpy.spin(node)
8:         node.destroy_node()
9:         rclpy.shutdown()
```

Každý uzol beží vo svojom vlastnom výpočtovom cykle. ROS2 poskytuje funkciu `rclpy.spin()`, ktorá udržiava uzol aktívny a zabezpečí, že môže reagovať na prichádzajúce správy alebo udalosti.

Témy

Témy sú asynchrónne komunikačné kanály, ktoré umožňujú uzlom vymieňať si správy. Každá téma má definovaný typ správy, ktorý určuje štruktúru posiadaných dát [23].

Publikovanie na tému: Uzol môže publikovať správy na konkrétnu tému. Publikované správy sú odosielané všetkým uzlom, ktoré sa na danú tému odoberajú.

Algorithm 2 Implementácia publikácie na Topic

```
1: class TalkerNode(Node):
2:     def __init__(self):
3:         self.counter = 0
4:         super().__init__('talker_node_py')
5:         self.publisher_ = self.create_publisher(String, 'my_topic', 10)
6:         self.timer_ = self.create_timer(0.5, self.publish_callback)
7:         self.get_logger().info("Talker Python node has been started")
8:     def publish_callback(self):
9:         msg = String()
10:        msg.data = f'Hello {self.counter}'
11:        self.publisher_.publish(msg)
12:        self.counter += 1
```

Odber na tému: Uzol má možnosť odberu témy, aby prijímal správy publikované inými uzlami. Pri prijatí správy sa spustí callback funkcia, ktorá spracuje prijaté dáta.

Algorithm 3 Implementácia odberu Topicu

```
1: def __init__(self):
2:     super().__init__("listener_node_py")
3:     self.get_logger().info("Listener Python node has been created")
4:     self.subscriber_ = self.create_subscription(String, 'my_topic',
        self.subscriber_callback, 10)
5:     def subscriber_callback(self, msg):
6:         data = msg.data
7:         self.get_logger().info(f'received data: {data}')
```

Služby (services)

Služby umožňujú synchrónnu komunikáciu medzi uzlami, kde jeden uzol poskytuje službu a druhý uzol posiela požiadavku na túto službu.

Vytvorenie služby: Uzol poskytujúci službu musí definovať callback funkciu, ktorá spracuje požiadavky a vráti odpoveď. [22]

Algorithm 4 Implementácia Service

```
1: class MinimalService(Node):
2:     def __init__(self):
3:         super().__init__('minimal_service')
4:         self.srv = self.create_service(AddTwoInts, 'add_two_ints',
            self.add_two_ints_callback)
5:     def add_two_ints_callback(self, request, response):
6:         response.sum = request.a + request.b
7:         self.get_logger().info('Incoming request\na: %d b: %d' % (request.a, request.b))
8:         return response
9: def main(args=None):
10:     rclpy.init(args=args)
11:     minimal_service = MinimalService()
12:     rclpy.spin(minimal_service)
13:     rclpy.shutdown()
14: if __name__ == '__main__':
```

Použitie služby: Uzol môže poslať požiadavku na službu a čakať na odpoveď. Používa sa na úlohy, ktoré vyžadujú okamžité spracovanie a odpoveď.

Akcie (actions)

Akcie sú rozšírením služieb a sú navrhnuté na vykonávanie dlhších úloh. Poskytujú mechanizmus na sledovanie priebehu úlohy a umožňujú posielat priebežné aktualizácie.

Vytvorenie akcie: Akcia pozostáva z dvoch hlavných častí: servera, ktorý vykonáva úlohu, a klienta, ktorý iniciuje požiadavku na úlohu. [22]

Algorithm 5 Akcia na generovanie Fibonacciho čísel

```
1: procedure EXECUTE_CALLBACK(goal_handle)
2:     sequence = [0, 1]
3:     for  $i = 2$  to goal_handle.request.order do
4:         sequence.append(sequence[i-1] + sequence[i-2])
5:     end for
6:     return Fibonacci.Result(sequence=sequence)
7: end procedure
```

Sledovanie akcie: Klient môže posielat požiadavky a prijímať spätné informácie o priebehu úlohy. To umožňuje komplexné spracovanie a interaktívne operácie.

Vývoj v ROS2

Vývoj začína vytvorením pracovnej plochy zahŕňajúcej zdrojové súbory, inštaláciu a spúšťanie aplikácií. Pracovná plocha sa delí na tri hlavné časti: **src**, **build**, a **install**. Zdrojové súbory sú organizované do balíkov (packages), ktoré môžu obsahovať kód uzlov, konfigurácie, a definície správ.

```
Workspace/  
|-- build/  
|-- install/  
|-- log/  
|-- src/  
    |-- my_package/  
        |-- launch/  
        |-- config/  
        |-- resource/  
        |-- test/  
        |-- package.xml  
        |-- setup.py
```

3.3 Základné komponenty a nástroje ROS2

Táto sekcia sa zaoberá základnými nástrojmi používanými v ROS2, ktoré sa zameriavajú nie len na samotný vývoj, ale aj vizualizáciu, modelovanie a simuláciu.

Rviz2 – vizualizácia dát a stavu robota

Rviz2 je vizualizačný nástroj, ktorý umožňuje zobraziť dáta zo senzorov a stav robota v reálnom čase.

Funkcie Rviz2

Rviz2 je súčasťou distribúcie ROS2 a je možné ho spustiť pomocou príkazu `rviz2`. Rviz2 poskytuje nasledujúce funkcionality:

- Vizualizácia 3D modelu robota.
- Zobrazovanie dát zo senzorov ako napríklad LIDAR alebo kamera.
- Vizualizácia plánovaných trás a interakcií s prostredím.

Gazebo simulátor

Gazebo využíva súbory vo formáte xacro na definovanie robotov, senzorov a prostredí. Tieto súbory špecifikujú fyzikálne vlastnosti, vizuálne prvky a správanie jednotlivých komponentov. Po zostavení a načítaní týchto súborov Gazebo spustí simuláciu.

Integrácia s ROS2

Pre ROS 2 Jazzy a novší simulátor *Gazebo Sim* (predtým Ignition) sa používajú balíky `ros_gz_sim`, `ros_gz_bridge` a súvisiace mosty namiesto staršieho balíka `gazebo_ros` z prostredia Gazebo Classic. Simulátor načíta SDF svet a model robota; senzory (kamera, laser) publikujú témy v priestore Gazebo, ktoré most prepojí na štandardné ROS 2 správy (`sensor_msgs`, `nav_msgs`, `tf2`).

Praktický príklad

V rámci vývoja sa simulácia spúšťa z pracovného priestoru s balíkmi `waver_sim` / `waver_nav`, napr. príkazom `ros2 launch waver_sim launch_sim.launch.py` alebo jednotným skriptom `run_sim.sh`, ktorý zdrojuje prostredie ROS 2 Jazzy, vyčistí staré procesy a voliteľne spustí teleop v samostatnom termináli. Parametre mosta (mapovanie tém Gazebo → ROS) sú v konfiguračnom súbore YAML; obraz z kamery je v ROS často dostupný na `/camera/image_raw`.

3.4 Integrácia senzorov a akčných členov

Táto sekcia pokrýva všeobecné princípy integrácie senzorov a akčných členov do robotických systémov s použitím ROS2.

Pripojenie senzorov a spracovanie dát

Senzory môžu byť pripojené k riadiacej jednotke cez rôzne komunikačné rozhrania. ROS2 balíky poskytujú ovládače, vďaka ktorým je ľahké pripojenie a konfigurácia senzorov.

Na asynchrónnu výmenu dát medzi senzorickými uzlami a ostatnými komponentmi systému sa využívajú vyššie spomenuté témy. Napríklad LIDAR môže publikovať dáta o vzdialenostiach na tému `/scan`, ktoré potom odoberajú uzly zodpovedné za mapovanie alebo vyhýbanie sa prekážkam.

Algorithm 6 Odber dát zo senzora

```
1: def callback(msg):  
2:     print("Received data: ", msg.data)  
3: subscription = create_subscription(SensorMsgType, 'sensor_topic',  
    callback, 10)
```

Kalibrácia a nastavenie

Senzory často vyžadujú kalibráciu a konfiguráciu, aby správne fungovali. ROS2 podporuje používanie parametrov a konfiguračných súborov, ktoré umožňujú dynamické nastavenie parametrov senzorov bez nutnosti zmeny kódu.

Súvis s príbuznými implementáciami (ROS2)

V prácach Skoleka [33] a Onderku [17] sa v praxi využívajú koncepty ROS2 priamo súvisiace s touto úlohou. Z prvej práce je pre nás užitočné: rozdelenie projektu do balíčkov podľa funkcie (senzory, riadenie, komunikácia), publikovanie dát zo senzorov (IMU, enkodéry, ultrazvuk) na témy a ich odber v riadiacom uzle, použitie launch súborov na spúšťanie celej sady uzlov a voľba QoS pre nízku latenciu alebo garancie doručenia. Z druhej práce vyplýva: distribuované spúšťanie systému (uzly na robotovi a na stacionárnom PC), využitie tf2 pre transformácie medzi rámami robota a senzorov, konfiguračné YAML súbory pre parametre uzlov, remapovanie tém v launch súboroch a prípadne rozšírenie o `ros2_control`, `slam_toolbox` alebo Nav2 pre pokročilú navigáciu. Obe práce ukazujú prepínanie medzi režimami (manuálny / autonómny) ako príklad dynamickej rekonfigurácie riadiaceho softvéru, čo je požadované aj v zadaní tejto práce.

micro-ROS a integrácia s embedded zariadeniami

micro-ROS [13] je rozšírenie ekosystému ROS 2 pre mikrokontroléry a embedded zariadenia s obmedzenou pamäťou a výkonom. Umožňuje, aby takéto zariadenia (napr. ESP32, STM32) vystupovali ako plnohodnotní účastníci ROS 2 siete: publikovali témy, odoberali témy, poskytovali alebo volali služby a pracovali s parametrami. Tým sa odstráni potreba vlastného protokolu (napr. HTTP alebo proprietárneho UART protokolu) medzi hlavným počítačom (Raspberry Pi) a riadiacou jednotkou na báze mikrokontroléra.

Funkčnosť a prepojenie s ROS 2. micro-ROS beží na embedded zariadení nad real-time operačným systémom (typicky FreeRTOS). Knižnica micro-ROS využíva rovnaké koncepty ako ROS 2 (DDS, témy, typy správ), ale s redukovanou závislosťou a pamäťovou náročnosťou. Komunikácia medzi mikrokontrolérom a zvyškom ROS 2 siete prebieha cez *micro-ROS agenta*, ktorý beží na Linuxe (napr. na Raspberry Pi alebo na vývojovom PC). Agent prijíma dáta z embedded zariadenia cez transport (sériový port UART, WiFi alebo Ethernet) a premostí ich do DDS siete. Uzly bežiace na RPi a uzly bežiace na mikrokontroléri tak vidia navzájom svoje témy v rámci jednej ROS 2 domény. micro-ROS je kompatibilný s distribúciami ROS 2 Humble a novšími (Iron, Jazzy) [14].

Vzorové uzly pre motor a IMU. Ako alternatíva k HTTP rozhraniu existuje ekosystém micro-ROS: mikrokontrolér môže cez agenta vystupovať priamo v ROS 2 sieti (témy, služby). V rámci tejto práce boli zvážené vzorové ukážky (adresár `micro_ros_examples`): subscriber na príkazy pohonu a publisher IMU. **V praktickej implementácii použitej v texte nižšie** je však primárne riešenie *HTTP JSON príkazy* medzi Raspberry Pi a firmwareom UGV Base General na ESP32 (pozri kapitolu 4), pretože priamo zodpovedá dokumentovanému API výrobcu dosky. Micro-ROS ostáva voľbou do budúcnosti pri požiadavke na jednotnú DDS doménu bez vlastného HTTP mosta; podrobné merania latencie agenta v tejto práci nie sú uvedené.

Kapitola 4

Implementácia

V tejto kapitole je popísaný návrh a priebeh implementácie v **logickom poradí vývoja**: najprv geometrický model a simulácia, potom fyzická zostava a manuálne riadenie, následne vrstvy autonómie (bludenie, SLAM, globálna navigácia Nav2 v simulácii, vizuálne sledovanie) a na záver alternatíva s ESP32/HTTP. Dve hardvérové zostavy (ESP32 + HTTP vs. Motor Driver HAT + senzory na RPi), Gazebo Sim a optický tok sú rozvedené v tomto usporiadaní.

Ciele implementácie v projekte. Implementácia v balíku **waverower** sleduje tieto zámerne rozhodnutia:

1. **Zladenie s konvenciami ROS 2** – štandardné typy správ, témy `/cmd_vel`, `/scan`, `/imu`, aby sa dal rovnaký kód pripojiť k hotovým nástrojom ako RViz, `slam_toolbox` alebo Nav2 pri integrácii v simulácii.
2. **Modularita** – samostatné uzly pre motor, LiDAR bludenie, optický tok, prepínanie režimov a teleprezenciu, aby bolo možné spúšťať len potrebnú podmnožinu pri ladení.
3. **Bezpečné správanie pri prepínaní** – zastavenie wanderu pri návrate do manuálu, oddelenie teleopu od autonómneho `/cmd_vel`.
4. **Opakovateľné nasadenie** – launch súbory, voliteľné systemd jednotky.
5. **Úspora času pri experimentoch** – simulácia v Gazebo pred jazdou na reálnom podvozku.

Text nižšie pri jednotlivých blokoch vždy uvádza, čo je implementované a *prečo* sa zvolila daná forma.

4.1 Postup implementácie v texte

Odporúčané čítanie sleduje skladanie systému rovnako ako pri ladení na reálnom robote: (1) definícia modelu v URDF/Xacro a **simulácia v Gazebo** s vizualizáciou v **RViz** (sek. 4.2); (2) popis platformy Wave Rover, senzorov na Raspberry Pi a **manuálneho** teleopu vrátane web rozhrania (sek. 4.3); (3) podrobnosti balíka **waverower** – motor, IMU, optický tok, **bludenie, SLAM, Nav2 (v simulácii), sledovanie lopty** (sek. 4.4); (4) prehľad blokov uzlov (sek. 4.5); (5) komunikácia s **UGV Base General** na ESP32 (sek. 4.6).

Tabuľka 4.1: Logické fázy implementácie: poradie, cieľ a väzba na text

Fáza	Čo sa rieši (výstup)	Prečo v tomto poradí
1	URDF/Xacro, Gazebo, RViz (sek. 4.2)	Geometria a TF sa overia bez rizika poškodenia hardvéru; rovnaký model slúži ako referencia pre montáž senzorov.
2	Zostava RPi + HAT + senzory, teleop, web UI (sek. 4.3)	Najprv spoľahlivý pohyb a základné témy; bez toho nemá zmysel ladžiť navigáciu.
3	Motor, IMU PID, optický tok, wander, SLAM, <code>ball_follow_standalone.launch.py</code> , CV (sek. 4.4); Nav2 len cez <code>waver_sim</code>	Vrstvy autonómie a korekcií stavajú na fungujúcom pohone a senzoroach; globálna navigácia Nav2 je v práci len v simulačnom workspace (<code>launch_sim.launch.py</code>), nie na fyzickom aute.
4	Prehľad uzlov a schéma (sek. 4.5)	Zhrnutie architektúry pre čitateľa a obhajobu návrhu.
5	ESP32 / HTTP alternatíva (sek. 4.6)	Dokumentácia druhej vetvy výrobcu; hlavná zostava ju nemusí používať.

4.2 Model robota, URDF/Xacro a simulácia v Gazebo a RViz

Pri modelovaní robota som použil formát Xacro. Celý model pozostáva z viacerých častí, vrátane podvozku, kolies a senzorov – LiDARu a kamery. Podvozok zodpovedá koncepcii diferenciálneho pohonu [31]. Komponenty sú cez `xacro:include` rozdelené do samostatných súborov: **účel** je jednoduchá úprava montážnych osov (napr. posun laseru) bez prepisovania celého URDF a znovupoužitie rovnakých fragmentov v simulácii aj pri generovaní TF pre RViz.

Hlavný súbor obsahuje odkazy na jednotlivé časti robota, ktoré sú pre prehľadnosť implementované v samostatných súboroch.

Definícia tela

Základný podvozok robota tvorí spoj (`fixed joint`) s jeho základňou (`base_link`). Podvozok je obdĺžnikový blok s definovanými vizuálnymi a kolíznymi vlastnosťami:

```
<link name="chassis">
  <visual>
    <origin xyz="0.15 0 0.02"/>
    <geometry>
      <box size="0.16 0.16 0.07"/>
    </geometry>
    <material name="white"/>
  </visual>
```

```

<collision>
  <origin xyz="0.15 0 0.02"/>
  <geometry>
    <box size="0.16 0.16 0.07"/>
  </geometry>
</collision>
<xacro:inertial_box mass="0.5" x="0.16" y="0.16" z="0.07">
  <origin xyz="0.15 0 0.02" rpy="0 0 0"/>
</xacro:inertial_box>
</link>

```

Výpis 4.1: Definícia podvozku

Kolesá boli pripojené k základni robota pomocou rotačných kĺbov (`continuous joint`). Každé koleso bolo definované ako valec s vizuálnymi a kolíznymi vlastnosťami.

Každý má definovaný materiál aj vlastnosti, ktorý Gazebo využíva na vizualizáciu a simuláciu.

Kamera a LIDAR

V súbore `camera.xacro` je definované, ako je kamera pripevnená na robotovi. Kamera je umiestnená na podvozku robota pomocou pevného kĺbu.

Kamera je pripojená na určité miesto na podvozku v definovaných súradniciach. Tento kód ukazuje, ako je to nastavené:

```

<joint name="camera_joint" type="fixed">
  <parent link="chassis"/>
  <child link="camera_link"/>
  <origin xyz="0.22 0 0.07" rpy="0 0 0"/>
</joint>

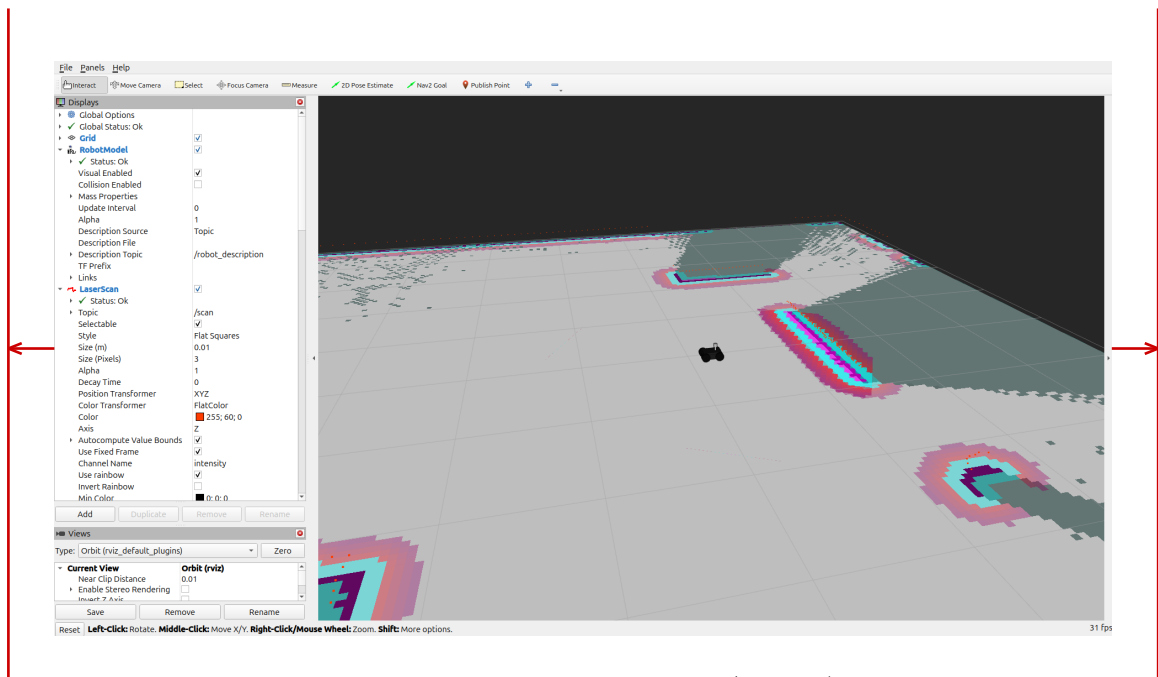
```

Rovnako ako kamera, aj LIDAR je pripojený k podvozku pomocou pevného kĺbu. Tento kĺb spája podvozok (link `chassis`) s rámom LIDARu (link `laser_frame`).

```

<joint name="laser_joint" type="fixed">
  <parent link="chassis"/>
  <child link="laser_frame"/>
  <origin xyz="0.1 0 0.07" rpy="0 0 0"/>
</joint>

```



Obr. 4.1: Model robota Wave Rover v RViz2 podľa URDF/Xacro (podvozok, kolesá, LiDAR, kamera)

Gazebo simulátor a RViz

V simulátore Gazebo Sim som v prostredí workspace pripravil model robota Wave Rover v súradniciach URDF/Xacro (podvozok, kolesá, LiDAR, kamera) a svet s prekážkami na demonštráciu senzorov a navigácie. [32] Sensory v simulácii sú napojené cez `ros_gz_bridge`: laser poskytuje dáta vhodné pre SLAM a Nav2, kamera obraz na tému kompatibilnú s vizualizáciou v RViz (napr. `/camera/image_raw`).

Spúšťanie: po zostavení workspace (`colcon build`) a zdrojovaní (`source install/setup.bash`) napr. `ros2 launch waver_sim launch_sim.launch.py`, prípadne skript `./run_sim.sh` v priečinku simulácie; ten môže spustiť teleop a bringup s Nav2 a `slam_toolbox`, ak sú príslušné balíky v workspace. RViz používa uloženú konfiguráciu (napr. `main.rviz` v `waver_sim`) so zobrazeniami mapy, laseru, plánu a obrazu z kamery.



Obr. 4.2: Simulovaný model Wave Rovera v prostredí Gazebo Sim

4.3 Aktuálna architektúra robota (autíčka)

Geometria podvozku, umiestnenie kolies, kamery a LiDARu v súboroch Xacro a správanie v Gazebo/RViz sú popísané v sekcii 4.2. Nasledujúci text sa vzťahuje na **fyzické zostavenie** platformy Wave Rover s Raspberry Pi, senzormi a motorovým HATom a na spôsob spúšťania manuálneho ovládania.

Aktuálny softvérový systém na platforme Wave Rover je implementovaný ako ROS 2 balík `waverover` (distribúcia Jazzy) na Raspberry Pi. Sekcia 4.4 popisuje **skutočnú implementáciu** v zdrojových kódach (C++ a Python) – motorový uzol, optický tok, IMU PID, rozdelenie LiDARu pri bludení a launch súbory – namiesto staršieho konceptu s ESP32 multiplexorom uvedeného nižšie pri iných uzloch.

Druhá hardvérová zostava: Motor Driver HAT a senzory na Raspberry Pi

Nezávisle od vyššie uvedenej vetvy s ESP32 sa v projekte používa konfigurácia, v ktorej Raspberry Pi priamo ovláda Waveshare *Motor Driver HAT* po zbernici I²C (typicky adresa 0x40) uzlami balíka `waverover`.

Implementačná trieda `MotorHatNode` (spustiteľná položka `waverover`) sa v launch súboroch pomenúva podľa kontextu, napr. `motor_hat_node` v súbore `manual_bringup.1 launch.py` alebo `wasd_motor_hat_node` v iných konfiguráciách. Uzol odoberá správy typu `geometry_msgs/msg/Twist` z témy určenej parametrami (teleop oproti `/cmd_vel`) podľa `control_mode`. Topic `/imu` dodáva skript `imu_serial_fusion_bridge.py` (MPU-6050 na mikrokontroléri cez USB; odôvodnenie v sekcii 4.6). LiDAR LD19 poskytuje `/scan` cez balík `ldlidar_ros2`. Táto zostava *nepoužíva* HTTP motorové príkazy na ESP32; pohon je výhradne cez HAT na RPi. Podrobný popis PID korekcie, optického toku a bludenia je v sekcii 4.4.

Komunikácia motor – LiDAR – IMU (ktoré fyzické rozhranie)

V ROS 2 sa dáta stretávajú na témach (`/cmd_vel`, `/scan`, `/imu`), ale na úrovni zapojenia medzi Raspberry Pi a perifériami platí:

- **Motory (Motor Driver HAT):** Raspberry Pi komunikuje s čipom PCA9685 na HAT-e cez zbernicu **I²C** (piny SDA/SCL na 40-pin konektore, typicky adresa 0x40). Uzol `MotorHatNode` cez ovládač I²C nastavuje PWM; HAT riadi H-mósty a prúd do štyroch motorov. **Motory teda nie sú na samostatnom UART-e**, ale za HATom.
- **LiDAR LD19:** senzor vystupuje ako **sériový UART** tok; v praxi je často pripojený cez **USB-UART** prevodník, takže na Linuxe pribudne zariadenie `/dev/ttyUSBn` (alebo ekvivalent). Balík `ldlidar_ros2` číta binárne rámce a publikuje `/scan`. Baud rate a port sú v launch parametroch drivera (typicky stovky kb/s podľa dokumentácie LD19).
- **IMU (MPU-6050 + Arduino Nano):** medzi Raspberry Pi a Nanom **nevedie I²C**. Na strane Nano je MPU-6050 na **lokálnej I²C** (len medzi čipom a Nanom); z Nano do RPi ide **USB** ako virtuálny sériový port (CDC), v systéme napr. `/dev/ttyACMn` alebo stabilná cesta `/dev/serial/by-id/...`. Skript `imu_serial_fusion_bridge.py` číta textový CSV stream (napr. 115200 Bd) a publikuje `/imu`.

Zhrnutie: **pohon = I²C** (RPi → PCA9685/HAT), **LiDAR = UART** (často cez **USB**) , **IMU = USB sériový link z Nano** (samotný MPU-6050 nie je na I²C priamo u RPi kvôli konfliktu s HATom, pozri sek. 4.6). Kamera (ak je v zostave) používa **CSI** (ribbon) alebo iné rozhranie podľa modulu; to je popísané v kapitole o `camera_ros`.

Tabuľka 4.2: Prepojenie periférií s Raspberry Pi: fyzické rozhranie a účel v ROS 2 (vetva Motor Driver HAT)

Subsystém	Fyzické rozhranie	Účel / typická ROS 2 téma
Pohon	I ² C na HAT (PCA9685)	<code>geometry_msgs/msg/Twist</code> → PWM; jednotný vstup pre teleop a autonómiu (wander; Nav2 len pri behu v simulácii)
LiDAR LD19	UART (často USB-UART)	<code>/scan</code> (<code>sensor_msgs/msg/LaserScan</code>) pre wander a SLAM; rovnaký typ správy využíva Nav2 v simulácii
IMU	USB sériové CSV z Arduino Nano	<code>/imu</code> pre PID na motore, otáčanie pri wanderi, EKF podľa launch
Kamera	CSI (Camera Module 3)	surový a komprimovaný obraz, teleprezencia, optický tok

Hlavné launch súbory balíka `waverower`

Pre opakovateľné spúšťanie je logika rozdelená do launch súborov (tabuľka 4.3). **Dôvod** je oddeliť skriptovanie od zdrojového kódu uzlov a mať jeden príkaz na konkrétny scenár (teleop, wander, mapovanie, sledovanie lopty). **Nav2 nie je súčasťou balíka waverower** — globálna navigácia v simulácii sa spúšťa z balíka `waver_sim` (súbor `launch_sim.launch.py`; sek. 4.2 a podsek. 4.4).

Webové rozhranie pre diaľkové ovládanie (teleprezencia z mobilu)

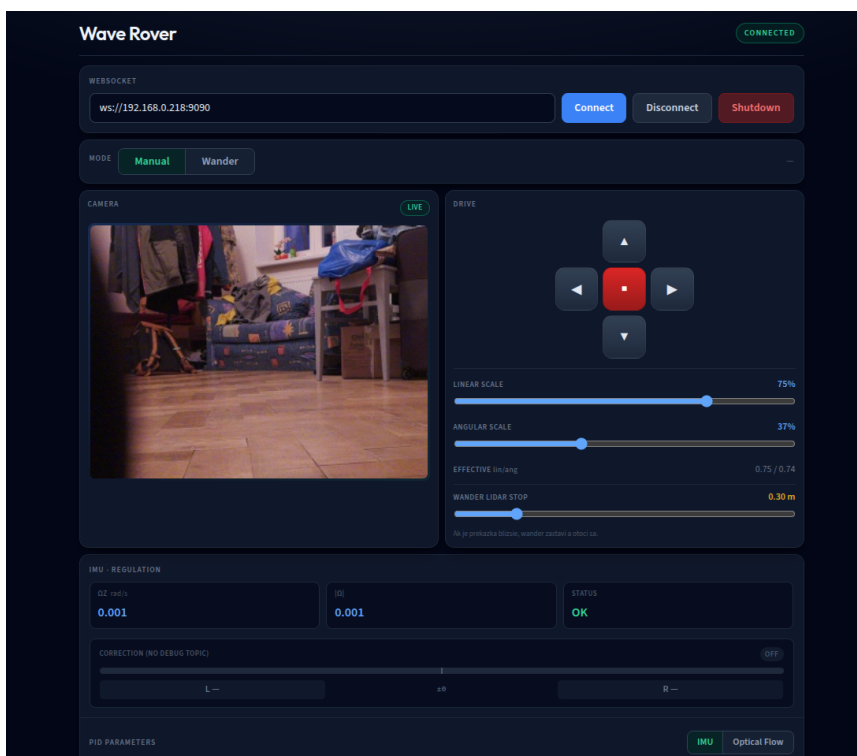
Pre ovládanie robota z prehliadača na telefóne alebo druhom PC je v balíku `waverover` pripravená jednoduchá webová aplikácia v priečinku `web_ui`. Statické súbory sú `index.html`, `app.js` a `style.css`.

Aplikácia používa knižnicu `roslibjs` [30], ktorá cez WebSocket komunikuje s balíkom `rosbridge_suite` [29]. Launch súbor `web_teleop_ui.launch.py` spúšťa `rosbridge_websocket` (port 9090) a jednoduchý HTTP server (`python3 -m http.server`) na rozhraní 0.0.0.0:8080, ktorý servíruje súbory z inštalovaného `share/waverover/web_ui/`.

Používateľ v lokálnej sieti otvorí stránku na `http://IP:8080` (kde `IP` je adresa Raspberry Pi). V rozhraní zadá WebSocket napr. vo formáte `ws://IP:9090`. Po pripojení klient vytvorí publisher na topic `/teleop_cmd_vel` typu `geometry_msgs/msg/Twist` — rovnaký kanál ako pri klávesnicovom teleope. Voliteľne odoberá komprimovaný obraz z kamery (predvolený typ správy je v kóde nastavený pre komprimovaný obraz; pri zmene kamery treba topic zladíť v `app.js`).

Tým sa dosiahne teleprezencia bez natívnej aplikácie: riadenie je v ROS 2, rozhranie je len tenká vrstva v prehliadači.

Prečo rosbridge a pevná frekvencia Twist. Rosbridge zjednodušuje prístup z ľubovoľného prehliadača bez natívneho ROS klienta; WebSocket je v lokálnej sieti primeraný kompromis medzi oneskorením a jednoduchosťou nasadenia. Periodické posielanie `Twist` pri nezmenenom vstupe (20 Hz) je zámerné: motorový uzol tak vždy dostáva čerstvú správu a pri výpadku spojenia sa dá detegovať *stagnácia* príkazov, čo znižuje riziko „nevedomého“ jazdenia po odpojení klienta.



Obr. 4.4: Webové rozhranie teleprezencie: pripojenie k `rosbridge`, náhľad z kamery a ovládanie pohybu

Zarovnanie jazdy: optický tok a IMU (stručný prehľad)

Pojem *zarovnanie* tu znamená **zníženie nechceného bočného driftu** pri jazde vpred. Používajú sa dva nezávislé prístupy (kamera vs. gyroskop); teoretický rámec optického toku (OpenCV, Lucas–Kanade, Farnebäck) je v [24], PID v zmysle spätnej väzby v [4]. **Implementačný detail** (presné názvy tém, parametre, podmienky zapnutia, plný PID vrátane integrálu a derivácie, rozdelenie LiDARu pri bludení) je sústredený v sekcii 4.4, aby sa v tejto podsekcii neopakovali celé odseky.

Súvis a obmedzenia. Optický tok a IMU korekcia riešia **rôzne príčiny** driftu (vizuálny posun scény vs. meraná yaw rate) a v launch súboroch sa dajú voliť oddelene (`use_flow`, `correction_mode`). Nezaručujú jazdu po priamke v mapových súradniciach bez globálnej navigácie alebo kvalitnej odometrie. V simulácii môže byť táto vrstva vypnutá.

4.4 Riadenie pohybu, optický tok, IMU PID a autonómne bludenie

Táto sekcia je hlavným technickým popisom balíka `waverower` v tejto práci. Zodpovedá súborom `motor_hat.cpp`, `optical_flow.cpp`, `optical_flow_dense.cpp`, `lidar_wander.py` a launch súborom uvedeným v tabuľke 4.3. Ďalej nasleduje popis Nav2 [18] pre **beh v simulácii** a referenčný príklad vizuálneho sledovania z repozitára *ball_tracker* [15]. **Poradie v texte** zodpovedá skladaniu systému: (1) motor a diferenciál, (2) IMU PID, (3) optický tok, (4) LiDAR a sektory pri bludení, (5) stavový automat bludenia, (6) SLAM, (7) Nav2 (simulácia), (8) farebná detekcia a sledovanie lopty.

Teória optického toku (OpenCV) je stručne v citácii v podsekcii vyššie; tu sú len súvislosti s ROS 2 a robotom.

Uzol motora a diferenciálny podvozok

Účel. Uzol motora je **jediný aktuátorový výstup** pre diferenciálny podvozok: prekladá abstraktný príkaz pohybu (ROS Twist) na PWM signály pre HAT. **Dôvod rozdelenia manual / auto** je bezpečnosť a predvídateľnosť: operátor musí mať výhradný kanál v manuáli, zatiaľ čo Nav2 (v simulácii), wander alebo sledovanie objektu posielajú príkazy len vtedy, keď je motor v automatickom režime — inak by sa zdroje príkazov bijú o jeden hardware.

Uzol `MotorHatNode` (spustiteľná položka `waverower`) riadi motorový HAT cez I2C (PCA9685). Pracuje v dvoch režimoch podľa parametra `control_mode`:

- **manual** – vstup z klávesnice (WASD) a zároveň odber `geometry_msgs/msg/Twist` z témy teleopu (predvolene `/teleop_cmd_vel`); hodnoty sa mapujú na *tank* riadenie (ľavé a pravé koleso), s časovým dobehom (*decay*) a vyhladením.
- **auto** – odber `geometry_msgs/msg/Twist` z témy `/cmd_vel` (názov sa dá zmeniť parametrom `cmd_vel_topic`); príkaz v , ω sa prepočíta na normované rýchlosti ľavého/pravého kolesa podľa rozchodu kolies a maximálnej lineárnej rýchlosti, s vyhladením a časovým limitom platnosti príkazu.

Finálne príkazy pre ľavý a pravý motor prechádzajú funkciou `apply_tank`, ktorá okrem mapovania na PWM aplikuje voliteľnú **korekciu podľa IMU** (pozri nižšie). Mapovanie

na *tank* riadenie (ľavé/pravé koleso) zodpovedá hardwareu štyroch motorov na podvozku a zjednodušuje ovládanie z jedného *Twist* bez modelu „ackermann“.

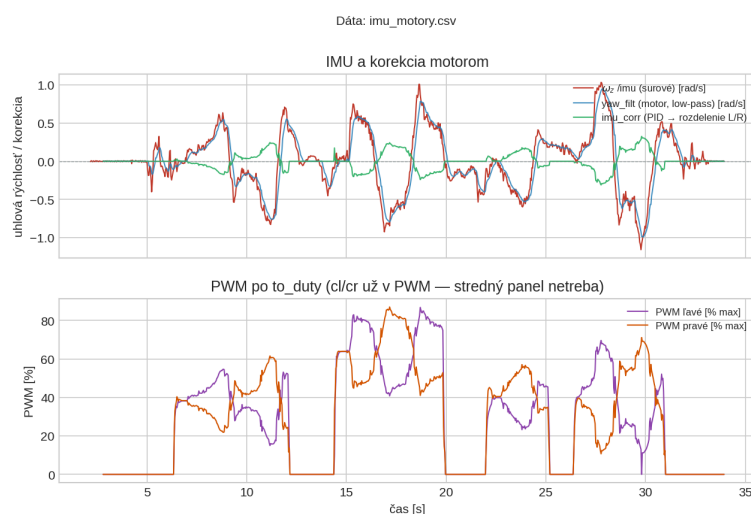
IMU a PID regulátor na osi yaw (kompenzácia driftu)

Prečo PID na motore. Robot nemá enkodéry kolies; pri jazde rovno sa prejavuje bočný drift. Ak je parameter `imu_correction` nastavený na `true`, uzol motora odoberá `sensor_msgs/msg/Imu` z témy `/imu` a ukladá z nej **uhlovú rýchlosť okolo osi z** (`angular_velocity.z`), t.j. okamžitú rýchlosť otáčania (yaw rate) z gyroskopu.

Regulátor je PID so zložkami K_p , K_i , K_d (parametre `imu_yaw_kp`, `imu_yaw_ki`, `imu_yaw_kd`). Definuje sa chyba ako samotná yaw rate (cieľ je jazdiť rovno, t.j. nulová otáčka okolo zvislej osi), pričom malé hodnoty pod prahom `imu_yaw_deadband` sa považujú za šum a nulujú. Integrálna zložka má obmedzenie (`imu_yaw_integral_limit`) kvôli anti-windup. Výstupom regulátora je korekčná hodnota `corr`, ktorá sa **odčíta od ľavého a pripočíta k pravému** normovanému príkazu kolesa, čím sa robot pri jazde vpred *dotočí* proti meranej rotácii.

Korekcia sa uplatňuje len vtedy, keď robot **nemá “nízku” doprednú rýchlosť** (v manuálnom režime pod prahom cca 0,12 v normalizovanom vstupe, v automate pod 0,02 m/s) – pri čistej rotácii na mieste alebo pri zastavení sa PID resetuje, aby neprepínal náhle pri zmene režimu.

Zdrojom `/imu` v projekte je typicky uzol `imu_serial_fusion_bridge.py`, ktorý číta CSV z mikrokontroléra (MPU-6050) cez USB a publikuje správy `Imu`.



ω_z (/imu): podľa ROS je +Z hore, $\omega_z > 0$ = proti SMH pri pohľade zhora (typ. doľava), $\omega_z < 0$ = so SMH (typ. doprava). Montáž IMU a imu_sign na motore môžu znamienko prevrátiť — over krúžením na mieste. PID: ľavé koleso -corr, pravé +corr.

Obr. 4.5: Príklad priebehu: uhlová rýchlosť z `/imu` a korekcia príkazu pohonu pri zapnutej IMU kompenzácií

Optický tok (korekcia dráhy podľa obrazu)

Účel oddeleného uzla. Na vyrovnanie jazdy sa používa **samostatný uzol**, ktorý neimplementuje PID, ale **priamu úpravu** požadovaného `angular.z` v správe `Twist` na základe horizontálneho posunu obrazu (optický tok). **Dôvod** je zachovať motorový uzol jednodu-

chý (PWM + voliteľná IMU) a dať vizuálnu vetvu možnosť vypnúť bez zásahu do nízkoúrovňového riadenia; výstup ide na samostatnú tému `/teleop_cmd_vel_corrected`, aby bolo v launchi jasné, či motor počúva surový teleop alebo už upravený príkaz. V bežnej konfigurácii sa používa **riedky Lucas–Kanade** (`optical_flow_node`): v predošlom snímku sa nájdu rohové body (`goodFeaturesToTrack`), medzi snímkami sa sledujú (`calcOpticalFlowPyrLK`) a z párov sa vypočíta priemer Δx . Experimentálne je v repozitári dostupný aj **hustý Farnebäck** ako samostatný uzol s `calcOpticalFlowFarneback`; do finálneho bringupu nemusí byť zaradený.

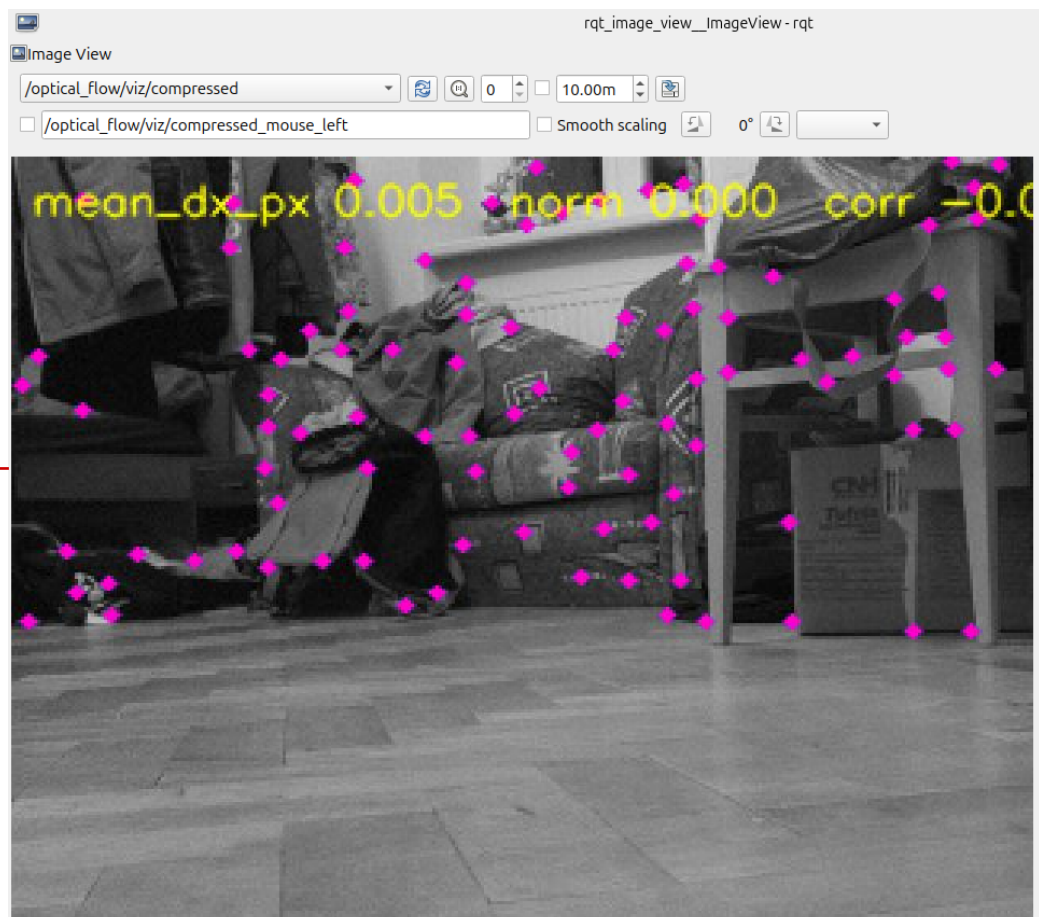
V oboch prípadoch sa priemer normalizuje šírkou snímky. V kóde platí: ak sa scéna v obraze posúva **dolava** (`mean_dx` záporné), robot v skutočnosti driftuje **doprava** — korekcia pridá kladné `angular.z` (otočenie dolava), aby sa drift kompenzoval. Výstup sa násobí `correction_gain` a obmedzuje na $\pm \text{max_correction}$. Uzol načítava komprimovaný obraz z kamery a posledný teleop Twist; timerom (cca 20 Hz) publikuje výsledok na tému `/teleop_cmd_vel_corrected`.

Korekcia z optického toku sa **pridá k** `angular.z` len ak súčasne platí: $|v_x| > \text{forward_threshold}$ (robot ide dopredu) a $|\omega_z| < \text{steer_deadzone}$ (používateľ aktívne “nezatáča”) – aby sa nebojoval s úmyselným zatáčaním.

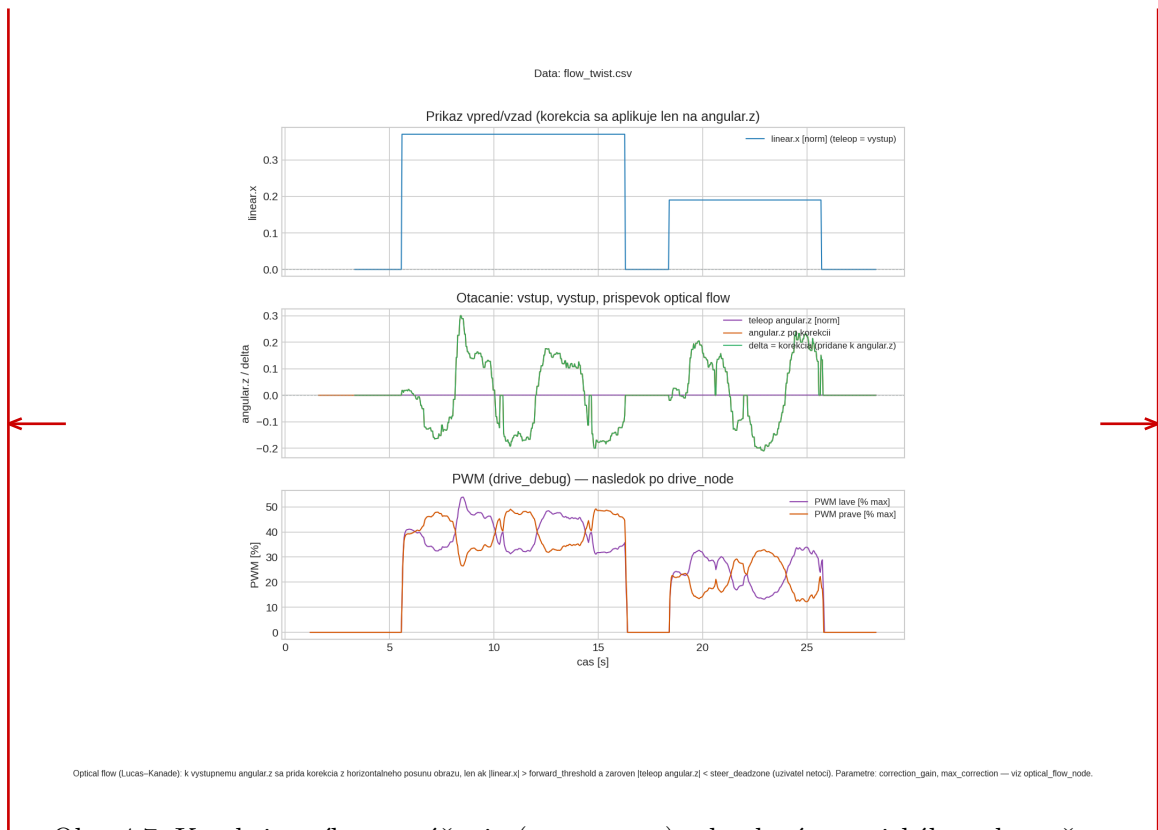
Parameter `correction_mode` v komplexnom launch (napr. `runtime_stack.launch.py`) určuje tri režimy korekcie driftu; motor v manuálnom režime odoberá buď priamy teleop, alebo `/teleop_cmd_vel_corrected`, ak je zapnutý optický tok. V `slam.launch.py` sa rovnaký argument forwarduje do vnoreného bringupu.

Tabuľka 4.4: Režimy `correction_mode`: čo kompenzujú a kedy dáva zmysel

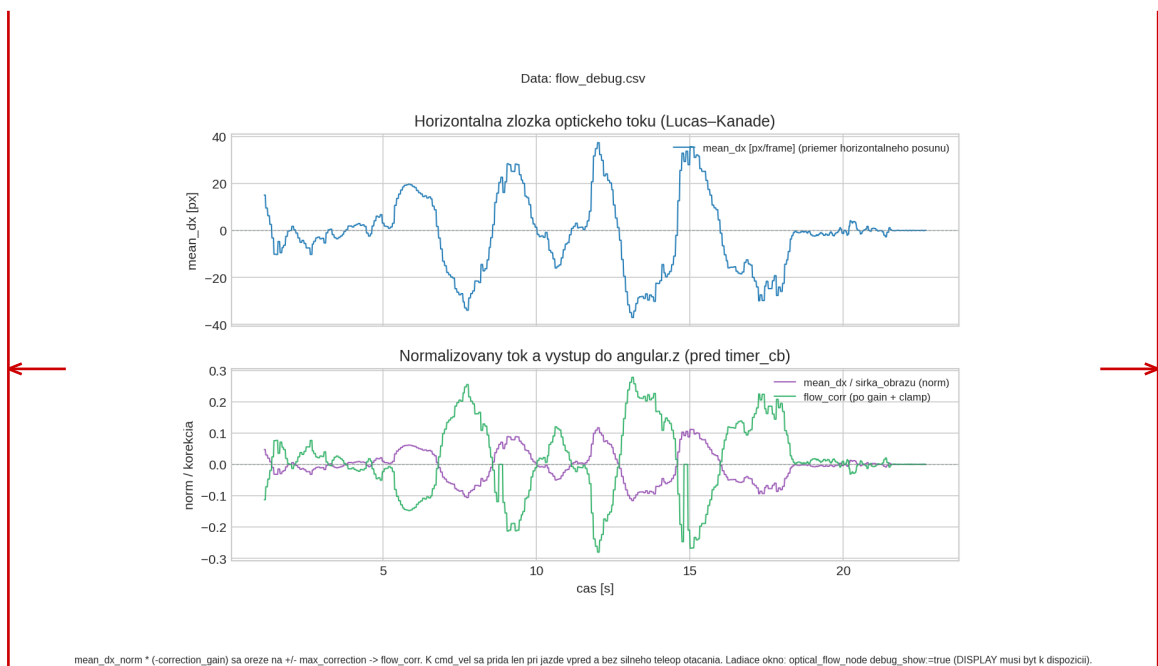
Hodnota	Účel a obmedzenia
<code>imu</code>	Kompenzácia rotácie podľa gyroskopu (yaw rate); funguje bez textúr v okolí; neopraví bočný posun bez ďalších senzorov.
<code>optical_flow</code>	Korekcia podľa vizuálneho posunu scény; vyžaduje dostatočný kontrast a osvetlenie; dopĺňa IMU pri inom type chyby dráhy.
<code>none</code>	Bez korekcie — baseline na porovnanie alebo keď šum/kalibrácia kompenzáciu zhoršujú.



Obr. 4.6: Debug výstup uzla optického toku (sledované body a odhad posunu scény mezi snímky)



Obr. 4.7: Korekcia príkazu otáčania ($angular.z$) odvodená z optického toku v čase



Obr. 4.8: Priemerná horizontálna zložka optického toku ($mean_dx$) – vstup do výpočtu korekcie

LiDAR LD19 a rozdelenie na sektory pri bludení

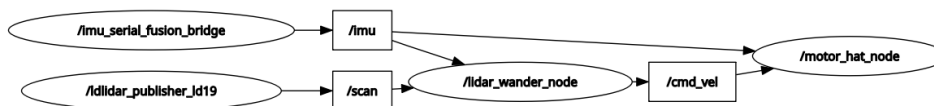
Driver `ldlidar_ros2` publikuje štandardnú správu `sensor_msgs/msg/LaserScan` na téme `/scan` a statickú transformáciu medzi `base_link` a rám laseru. Pre **autonómne blude-**

nie skript `lidar_wander.py` nepracuje s jedným “front” kuželom, ale delí priestor v **ráme robota** na tri významové oblasti po obvodu (tabuľka 4.5). **Dôvod** je jednoduché rozhodovacie pravidlo: namiesto drahého plánovača stačí porovnať minimálne vzdialenosti v troch oblúkoch a vybrať smer otočenia.

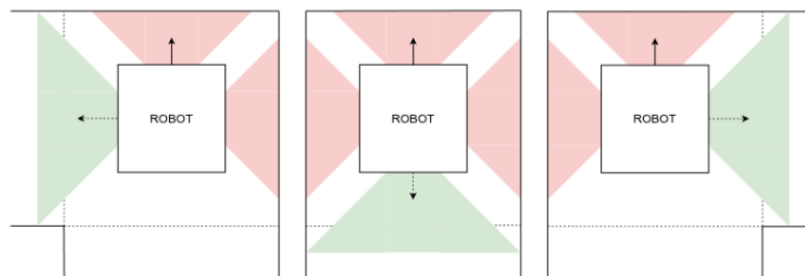
Tabuľka 4.5: Rozdelenie laserového skenu na sektory v `lidar_wander` (uhly v ráme robota)

Sektor	Uhol od	Uhol do
vpred	-45°	$+45^\circ$
vľavo	$+45^\circ$	$+135^\circ$
vpravo	-135°	-45°

Pre každú oblasť sa z aktuálneho poľa **ranges** vypočíta **minimálna** platná vzdialenosť (ignorujú sa neplatné alebo príliš krátke hodnoty). Parameter `lidar_rotation_deg` otočí celý laserový sken voči rámu robota (napr. ak je fyzicky montáž otočená o 180°), aby “vpred” zodpovedalo skutočnému smeru jazdy.



Obr. 4.9: Laserový sken v `rqt` pri autonómnom bludení (`wander.launch.py` alebo režim `wander` v `runtime_stack`); doplnok k grafu uzlov na obr. 4.3, kde sú uzly `lidar_wander_node`, driver LD19 a motor v `auto`



Obr. 4.10: Zjednodušený blokový diagram logiky bludenia (rozhodovanie podľa vzdialeností v sektoroch)

Autonómne bludenie (`lidar_wander.py`) a stavový automat

Prečo jednoduchý stavový automat. Implementácia bludenia zámerne **nevyžaduje mapu ani Nav2**: cieľom je ukázať reaktívne správanie v reálnom čase len z LiDARu a IMU, čo stačí na demonstráciu autonómie na klzkom podvozku bez enkodérov a zároveň znižuje výpočtovú záťaž oproti plnému stacku navigácie.

Uzol `lidar_wander_node` publikuje `geometry_msgs/msg/Twist` na `/cmd_vel` (názov je parametrizovateľný). Interný automat má stavy **jazda vpred** a **otáčanie**:

1. V predvolenom stave robot žiada konštantnú doprednú rýchlosť `forward_speed`, ak je minimálna vzdialenosť vpredu väčšia ako prah `threshold_m`.
2. Ak je vpredu prekážka bližšie ako prah, uzol porovná voľnosť vľavo a vpravo. Podľa toho spustí otočenie o 90° vľavo alebo vpravo; ak sú obe strany voľné, vyberie stranu s väčšou vzdialenosťou; ak sú obe zablokované, otočenie o 180° .
3. Počas otočenia sa uplatňuje konštantná uhlová rýchlosť `turn_speed`; **uhol otočenia** sa sleduje integráciou $|\omega_z|$ z IMU v čase (odber `/imu`) a porovnaním s cieľovým uhlom; pri prekročení času `turn_timeout_s` sa otočenie ukončí aj bez dosiahnutia presného uhla (záchranný timeout).

Launch súbor `wander.launch.py` spúšťa motor v režime `auto`, zapína IMU most, statický TF `base_link` $\rightarrow$ `imu_link`, driver LD19 a uzol `wander` – t.j. kompletný reťazec pre jazdu bez mapy a bez Nav2.

Prepínanie prevádzkových režimov za behu (`robot_mode_switch.py`)

Systém podporuje prepínanie medzi manuálnym a autonómnym režimom bez reštartu akéhokoľvek uzla. Prepínač je implementovaný ako samostatný ROS 2 uzol `robot_mode_switch` (súbor `waverover/scripts/robot_mode_switch.py`), ktorý po spustení poskytuje služby typu `std_srvs/srv/Trigger`:

- `/waverover/switch_to_manual` – prepne do manuálneho režimu,
- `/waverover/switch_to_wander` – prepne do autonómného bludenia,
- `/waverover/shutdown` – požiadavka na **vypnutie počítača** (Raspberry Pi); mechanizmus je popísaný nižšie.

Tabuľka 4.6: Logické vrstvy konfigurácie: prevádzkový režim vs. zdroj príkazu pohybu vs. korekcia

Vrstva	Popis a účel
Motor control_mode	manual : výhradne <code>/teleop_cmd_vel</code> (bezpečný teleop). auto : <code>/cmd_vel</code> z wanderu, z Nav2 pri simulačnom behu alebo z iného autonómného zdroja.
Prepínač robot_mode_switch	Službami sa prepína medzi manuálom a wanderom; zastavenie autonómie bez reštartu uzlov (sek. 4.4).
Launch correction_mode	IMU vs. optický tok vs. vypnuté (tab. 4.4); nezávislé od prepínania wander/manuál.
SLAM / Nav2	SLAM: <code>slam.launch.py</code> (tab. 4.3) — mapovanie na hardvéri aj v simulácii. Nav2: <i>nie</i> v <code>waverover</code> , len simulačný balík <code>waver_sim</code> (<code>launch_sim.launch.py</code>).

Mechanizmus prepnutia. Prepínanie sa opiera o vzdialenú dynamickú rekonfiguráciu parametrov prostredníctvom rozhrania `AsyncParameterClient` (trieda dostupná v distribúcii ROS 2 Jazzy; pre Humble existuje alias `AsyncParametersClient`). Tento klient umožňuje asynchrónne zavolať štandardnú ROS 2 parametrovú službu `set_parameters` na ľubovoľnom bežiacom uzle bez priameho prepojenia v zdrojovom kóde.

Pri prepnutí do **manuálneho režimu** sa sekvenčne nastaví dva parametre:

1. `motor_hat_node: control_mode` $\leftarrow$ "manual" – motorový uzol prestane sledovať `/cmd_vel` (autonómny zdroj) a začne reagovať výhradne na príkazy z `/teleop_cmd_vel`.
2. `lidar_wander_node: enabled` $\leftarrow$ False – riadiaci cyklus bludenia sa okamžite zastaví; uzol publikuje nulový Twist, čím sa robot bezpečne zastaví.

Pri prepnutí do **autonómneho režimu** sa postupuje opačne: motorový uzol dostane `control_mode` $\leftarrow$ "auto" a uzlu bludenia sa nastaví `enabled` $\leftarrow$ True, čím sa aktivuje jeho riadiaci cyklus.

Problém súbežnosti a riešenie deadlocku. Callbacks ROS 2 služieb bežia predvolene v `MutuallyExclusiveCallbackGroup`, ktorý zakazuje súbežné vykonávanie viacerých callbackov toho istého uzla. Keby servisný callback čakal na výsledok `set_parameters` (volanie inej služby) v tomto nastavení, vznikol by deadlock: odpoveď parametrovej služby by nemohla byť doručená, pretože executor by čakal na uvoľnenie zámku, ktorý sám drží.

Riešením je kombinácia `ReentrantCallbackGroup` a `MultiThreadedExecutor` so štyrmi vláknami. `ReentrantCallbackGroup` dovoľuje vstup do callbacku aj vtedy, keď iná inštancia toho istého callbacku ešte beží; viac vlákien executora zabezpečí, že odpoveď vzdialenej parametrovej služby bude doručená, aj keď hlavný servisný callback čaká. Aktívne čakanie na výsledok `Future` je implementované pomocnou metódou `_wait_future`, ktorá vo slučke 5 ms kontroluje stav s celkovým limitom 10 s – po uplynutí limitu sa volanie považuje za neúspešné a service vráti `success = False`.

Po dokončení oboch vzdialených volaní uzol overí výsledky cez pole `results` každého `Future`; ak niektoré nastavenie zlyhalo (`successful = False`), vráti chybovú odpoveď. V prípade úspechu vráti `success = True` a reťazec s názvom aktívneho režimu.

Integrácia v launch súboroch. Uzol `robot_mode_switch` je súčasťou komplexného launch súboru `runtime_stack.launch.py`, ktorý pri štarte celého systému spúšťa motorový uzol, uzol bludenia aj prepínač v jednom procese. Počiatočný režim je určený argumentom `stack_mode` (manual alebo wander); zodpovedajúce parametre sa odvodí priamo v launch súbore a uzly štartujú v správnom stave bez toho, aby bolo nutné volať prepínanie služby ihneď po štarte.

Webové ovládacie rozhranie – implementačné detaily

Webové rozhranie (`waverover/web_ui/`) je jednostránková aplikácia tvorená súborami `index.html`, `app.js` a `style.css`. Súbory sú servírované HTTP serverom spusteným v launch súbore `web_teleop_ui.launch.py` na porte 8080. Pre prístup z telefónu stačí v rovnakej sieti otvoriť v prehliadači `http://IP:8080` (kde *IP* je adresa Raspberry Pi).

Komunikačná vrstva. JavaScript využíva knižnicu `roslibjs`, ktorá nadväzuje WebSocket spojenie s `rosbridge_websocket` bežiacim na porte 9090. Po úspešnom pripojení aplikácia

vytvorí publisher na topic `/teleop_cmd_vel` (typ `geometry_msgs/msg/Twist`) a spustí časovač s frekvenciou 20 Hz. Správy sa publikujú pravidelne bez ohľadu na zmenu vstupov, čím sa predchádza situácii, keď motorový uzol nevie, či operátor drží tlačidlo alebo spojenie vypadlo.

Paralelne sa aktivuje subscriber na `/camera/camera_node/image_raw/compressed`. Prichádzajúce Base64-enkódované JPEG snímky sa dekodujú a vkladajú do elementu `<img>` s obmedzením frekvencie na ≈ 12 snímok za sekundu (filter 80 ms).

Pohybové príkazy a dotykové ovládanie. Každé smerové tlačidlo D-padu nesie HTML atribúty `data-lx` a `data-az` s normalizovanými hodnotami lineárnej a uhlovej rýchlosti. Tieto hodnoty sú škálované konštantami `MAX_LINEAR = 0,45 m/s` a `MAX_ANGULAR = 1,0 rad/s`. Na mobilných zariadeniach sú registrované udalosti `touchstart` (okamžité nastavenie rýchlosti, `preventDefault` zabráňuje emulovanému `click`) a `touchend` (nulová rýchlosť); pre stolné počítače sú použité ekvivalentné udalosti myši `mousedown`, `mouseup` a `mouseleave`.

Prepínanie režimov z rozhrania. Tlačidlá *Manual* a *Wander* volajú ROS 2 služby `/waverower/switch_to_manual` a `/waverower/switch_to_wander` (sekcia 4.4) prostredníctvom objektu `ROSLIB.Service`. Volanie je asynchrónne: tlačidlá sa počas čakania na odpoveď deaktivujú atribútom `disabled`, aby sa zabránilo opakovaným požiadavkám. Po prijatí odpovede sa vizuálny stav tlačidiel aktualizuje podľa výsledku; aktívny režim je zvýraznený zelenou farbou (CSS trieda `active`).

Na mobilných prehliadačoch môže byť udalosť `click` potlačená v dôsledku `preventDefault` na iných dotykových elementoch. Problém sa rieši registráciou udalosti `touchend` priamo na každom tlačidle prepínača s vlastným `preventDefault`, čím sa zaručí okamžitá odozva aj na dotykovom zariadení.

Vzdialené vypnutie Raspberry Pi (`/waverower/shutdown`). Pre vypnutie robota bez fyzického prístupu je v tom istom uzle `robot_mode_switch` služba `/waverower/shutdown`. Po zavolaní **najprv vráti úspešnú odpoveď** (`success=True`), aby klient cez `rosbridge` stihol prijať odpoveď skôr, než systém padne. Následne sa v samostatnom vlákne spustí časovač (oneskorenie približne 1 s) a vykoná sa `sudo systemctl poweroff`. Na pracovnej stanici musí byť pre používateľa ROS povolené spustenie `poweroff` bez hesla (napr. pravidlo v `sudoers`) alebo ekvivalent.

Webová časť (`connection-mode.js`) pred volaním zobrazí potvrdenie v prehliadači; po súhlase zavolá funkciu `rosTrigger` z `ros.js`, ktorá odošle požiadavku na službu typu `std_srvs/Trigger`. Tlačidlo *Shutdown* je dostupné len pri aktívnom WebSocket spojení. Pri odovzdávanej verzii práce je vhodné voliteľne doplniť snímku potvrdzovacieho dialógu pred vypnutím.

Vizuálny dizajn. Rozhranie používa tmavú farebnú schému optimalizovanú pre čitateľnosť v exteriérových podmienkach. Meta tag `viewport` s parametrami `maximum-scale=1`, `user-scalable=no` zabráňuje neúmyselnému priblíženiu pri dotyku a eliminuje 300 ms oneskorenie kliknutia prítomné v starších mobilných prehliadačoch. Tlačidlám je nastavená CSS vlastnosť `touch-action:manipulation`, ktorá ďalej skracuje dotykovú latenciu. Na telefóne alebo tablete sa zobrazuje tá istá webová stránka ako na počítači; prvky sa prispôbia šírke obrazovky, takže samostatný snímok mobilného rozhrania nie je v práci nutný (ilustrácia desktopového náhľadu je na obr. 4.4).

Automatické spúšťanie a ochrana procesov (systemd)

Pre spoľahlivú prevádzku robota bez manuálneho zásahu sú kritické uzly registrované ako systemd jednotky, čo zaručuje ich automatické spustenie po naboťovaní Raspberry Pi a opätovné spustenie po páde procesu.

Balík `waverower` obsahuje v adresári `systemd/` dve unit súbory:

- **ld19-lidar.service** – spúšťa driver LiDARu LD19 cez launch súbor `ldlidar_ros2_ld19.launch.py`; publikuje topic `/scan`.
- **waverower.service** – spúšťa motorový uzol `waverower_motor` priamo cez `ros2 run`; závisí od `ld19-lidar.service` a voliteľne od `camera-ros.service`.

Obidva unit súbory zdieľajú rovnakú štruktúru:

1. **Závislosť na sieťovom rozhraní** (`After=network.target`, `Wants=network-online.target`) – zaistí, že ROS 2 DDS multicast je k dispozícii pred štartom uzlov.
2. **Sourcing prostredia** – v `ExecStart` sa cez `bash -c` explicitne spúšťa načítanie skriptov ROS 2: najprv `/opt/ros/jazzy/setup.bash`, potom `install/setup.bash` vo workspace (premenná prostredia `WS`). Systemd nezdedí používateľský shell, takže bez toho by príkazy `ros2` neboli v `PATH`.
3. **Automatický reštart** (`Restart=always`, `RestartSec=5`) – ak uzol skončí z akéhokoľvek dôvodu (výnimka v Pythone, odpojenie I²C zbernice, zabitie procesom OOM killer), systemd ho po 5 sekundách znova spustí bez nutnosti zásahu operátora.
4. **Logovanie** (`StandardOutput=journal`, `StandardError=journal`) – výstup všetkých uzlov je dostupný cez `journalctl -u waverower.service -f`, čo uľahčuje ladenie aj pri behu na pozadí bez terminálu.

Nasadenie unit súborov na robot sa vykonáva skopírovaním do `/etc/systemd/system/` a príkazmi:

```
sudo systemctl daemon-reload
sudo systemctl enable ld19-lidar.service
sudo systemctl enable waverower.service
sudo systemctl start ld19-lidar.service
sudo systemctl start waverower.service
```

Okrem systemd ochrany obsahuje aj spúšťací skript `waverower_manual_teleop.sh` vlastnú obranu pred neporiadnym stavom prostredia. Na začiatku skript ukončí všetky prípadne bežiacie inštancie uzlov rovnakých mien (`kill -f`) – zamedzí tak situácii, keď predchádzajúce spustenie ostalo visieť v pamäti. Pre bezpečné čistenie pri ukončení je zaregistrovaná Bash trap na signály `EXIT`, `INT` a `TERM`: `cleanup()` zabije `teleop_twist_keyboard` aj motorový uzol (uložené PID v premennej `WAVER_PID`) a počká na ich ukončenie – robot tak nezostane nekontrolovateľne pohybovať sa po prerušení SSH spojenia alebo `Ctrl+C`.

Integrita súborového systému pri výpadku napájania, kontajnery a súvis so zadaním

Oficiálne zadanie bakalárskej práce (formulácia v češtine) okrem iného uvádza:

Posledním problémem k řešení je zajistit, aby při vypnutí RPi (bez regulérního shutdownu) nedošlo k porušení integrity filesystemu (read-only fs plus vhodné použití kontejneru), případně hardwarově zajistit regulérní shutdown při požadavku na vypnutí.

Regulérny shutdown na požiadanie. Túto časť zadania adresuje **softvér** v tomto projekte: služba `/waverower/shutdown` (uzol `robot_mode_switch`, sek. 4.4; ovládanie z Web UI vyššie) po krátkom oneskorení zavolá `sudo systemctl poweroff`, čo predstavuje **regulérny shutdown** operačného systému, ak ho operátor použije pred odpojením napájania. Hardvérové riešenie (UPS, GPIO tlačidlo, HAT s riadením napájania) v repozitári nie je implementované; slovo *případně* ho ponecháva ako voliteľné rozšírenie.

Read-only súborový systém a náhle vypnutie. Ochrana integrity koreňového súborového systému pri úplnom výpadku napätia (bez shutdownu) sa v praxi rieši **konfiguráciou OS** na Raspberry Pi (napr. režim Overlay v Raspberry Pi OS, prípadne obdobné opatrenie na Ubuntu), nie zdrojovým kódom uzlov ROS 2. Bez takejto konfigurácie zostáva riziko poškodenia oddielu pri odpojení kábla; používanie regulérneho vypnutia ho znižuje, ale nenahrádza read-only ani overlay. Podrobný návod, tabuľka možností a mapovanie jednotlivých viet zadania na riešenie sú v súbore `waverower/docs/fs-a-kontejnery.md` v balíku `waverower`; po `colcon build` a inštalácii je k dispozícii aj pod `install/share/waverower/docs/`.

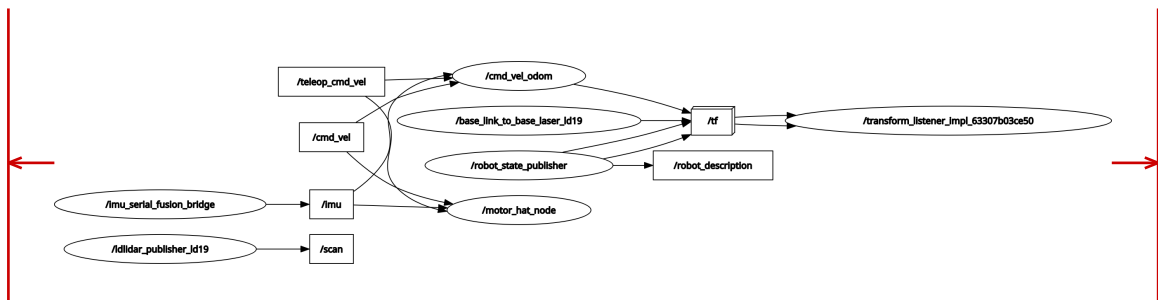
Kontajnery (Docker / Podman). ROS 2 stack na robote s prístupom k I²C (Motor HAT), USB sériovým rozhraniam (LiDAR, IMU) a kamere je v projekte prevádzkovaný **natívne**. Kontajnery sú pre túto platformu vhodnejšie napr. na vývojársky počítač (oficiálne obrazy ROS) alebo voliteľné izolované služby; nie sú povinnou formou nasadenia na RPi. Tým je splnený zmysel požiadavky na *vhodné použitie kontejneru* vo forme **odôvodnenej voľby** prostredia (Pi = natívne beh, PC = voliteľné kontajnery), nie nutne beh celého stacku v kontajneri na robotovi.

Zhrnutie pre obhajobu. Splnenie tejto časti zadania spočíva v kombinácii: (1) aplikovaná stratégia read-only/overlay **na cieľovom Raspberry Pi** (mimo samotného git repozitára) a jej stručný popis v texte práce; (2) uvedená argumentácia kontajnerov; (3) regulérne vypnutie cez `/waverower/shutdown` a overené právo spustiť `poweroff` (napr. `sudoers`).

Prepojenie s mapovaním (SLAM)

Účel SLAM v práci. Pre tvorbu 2D mapy interiéru sa používa `slam_toolbox` namiesto vlastného SLAM algoritmu: **dôvodom** je overená integrácia s ROS 2, podpora asynchrónneho mapovania a kompatibilita s Nav2 v simulácii (rovnaká mapa bez prepisovania kódu).

Pre režim SLAM (`slam.launch.py`) sa okrem vyššie uvedených uzlov spúšťa `slam_toolbox` s parametrami z `slam.yaml`; voliteľne `robot_localization` (EKF) podľa `ekf.yaml`. Tu LiDAR slúži na **celý 360° scan** pre výstavbu mapy, zatiaľ čo logika `lidar_wander` používa ten istý `/scan`, ale len agregáciu do troch sektorov pre rozhodovanie o smere – ide o **dva rôzne účely** jedného senzorového toku.



Obr. 4.11: Graf uzlov a tém (`rqt_graph`) pri manuálnom ovládaní a súčasnom behu SLAM (`slam.launch.py` so senzormi a `slam_toolbox`)

Globálna navigácia Nav2 (simulácia, balík `waver_sim`) a rozdiel voči bludeniu

Obmedzenie rozsahu. Nav2 **nie je** launch súborom v balíku `waverower`. V repozitári sa globálna navigácia spája so simuláciou cez balík `waver_sim` a súbor `launch/launch_sim.launch.py`. Na fyzickom Wave Roveri s Raspberry Pi bol Nav2 v rámci tejto práce *nenasadený a neoverovaný* — autonómia na hardvéri spočíva v bludení, manuálnom riadení a mapovaní (SLAM), nie v plnom Nav2.

Autonómne bludenie (`lidar_wander.py`) je *reaktívne* správanie bez mapy a bez globálnej plánovacej vrstvy: rozhoduje len z aktuálneho LiDARu a IMU. **Nav2** (Navigation 2) [18] je naopak stack pre navigáciu v známej alebo sa práve budovanej **mape** v súradniciach `map` → `odom` → `base_link`, s plánovačom trasy, obmedzeniami rýchlosti a správaniami pri zaseknutí.

Prečo Nav2 navyše oproti wanderu. Cieľom integrácie Nav2 v simulácii nie je nahraďiť jednoduché bludenie, ale ukázať **plnohodnotnú globálnu navigáciu** v mapovanom prostredí: plán v mape, obchádzanie prekážok v costmapách a regulátor sledovania trajektórie. V simulácii poskytuje Gazebo odometriu a `tf` cez most `ros_gz_bridge` (konfigurácia `waver_sim/config/ros_gz_bridge.yaml`), takže nie je potrebná vlastná integrácia `/cmd_vel` do `odom` ako pri reálnom podvozku bez enkodérov.

Čo spúšťa `launch_sim.launch.py`. Launch v `waver_sim` (voliteľne cez skript `run_sim.sh` v simulačnom workspace) zostavuje okrem iného:

- **Gazebo Sim a robot** – `gz_sim` so svetom z balíka `waver_gazebo`, uzol `create` spawne model podľa témy `robot_description`; `robot_state_publisher` generuje TF z URDF/Xacro z `waver_sim`.
- **Most a riadenie** – `ros_gz_bridge` (`ros_gz_bridge.yaml`) pre `/clock`, `/cmd_vel`, `/odom`, `/tf`, `laser` atď.; uzol `twist_mux` zlučuje viacero zdrojov `/cmd_vel` podľa priority (`twist_mux_params.yaml`).
- **Nav2 a SLAM Toolbox** – oneskoreným `IncludeLaunchDescription` sa spustí štandardný `bringup_launch.py` z balíka `nav2_bringup` s argumentom `slam:=True`, parametrami Nav2 v `waver_sim/config/nav2_params.yaml` a SLAM v `waver_sim/config/slam_params.yaml`.
- **RViz2** – neskorším časovačom sa otvorí uložená konfigurácia `waver_sim/config/main.rviz`.

Ciele sa dajú zadať typicky z RViz (*2D Nav Goal*) alebo ďalšími rozhraniami podľa dokumentácie Nav2; podrobnosti parametrov sú v uvedených YAML súboroch. Pre úvod do architektúry a pojmov (plánovač, regulátor, costmapy) slúži oficiálna dokumentácia [18].

Zhrnutie. Uvedený reťazec bol v práci overený v simulácii. Na fyzickom podvozku nie je Nav2 v rámci bakalárskej práce spúšťaný ani hodnotený.

Príklad CV: sledovanie farebného objektu (repozitár `ball_tracker`)

Ako **ilustrácia** klasického postupu počítačového videnia pre mobilnú robotiku (bez učenia) slúži verejný ROS 2 balík od Josha Newansa [15]. V histórii repozitára posledné tri commity zodpovedajú: (1) počiatočné pridanie uzlov `detect_ball`, `follow_ball`, launch a konfigurácie; (2) súbor `process_image.py` s kompletným spracovaním obrazu; (3) drobná oprava v `README.md`. Autor uvádza inšpiráciu tutoriálom Tiziana Fiorenzaniho pre ROS 1 a upozorňuje, že pri `colcon build --symlink-install` sa launch a YAML po zmene treba znova zbuildovať.

Reťazec spracovania v `find_circles (process_image.py)`. Vstup je snímok v BGR (`cv_bridge` z témy `/image_in`). Postup je tento:

1. **Vyhľadanie** – `cv2.blur` s malým jadrom, aby sa potlačil šum pred prahovaním.
2. **Obmedzenie oblasti záujmu** – obdĺžnik `search_window` zadaný percentuálne (`x_min` až `y_max`) sa premení na pixely a aplikuje ako maska (`apply_search_window`), aby sa detekcia sústredila na časť snímku.
3. **Konverzia do HSV a prahovanie** `cv2.inRange` podľa parametrov `h_min/h_max`, `s_*`, `v_*` – výsledkom je binárna maska farebnej triedy (typicky farebná lopta na jednofarebnom pozadí).
4. **Morfológia** – `dilate` a `erode` (po 2 iteráciách) vyplnia diery a ohľadia okraje masky.
5. **Inverzia** (`255 - mask`) pred detekciou blobov, aby algoritmus hľadal tmavé objekty na svetlej ploche v zmysle nastavenia `SimpleBlobDetector`.
6. **OpenCV SimpleBlobDetector** – z parametrov vyplýva filtrovanie podľa plochy (`minArea/maxArea`), circularity, konvexity a inertia ratio; nájdu sa kľúčové body (*keypoints*) s polohou a veľkosťou.
7. **Ďalšie filtrovanie** podľa relatívnej veľkosti (`sz_min`, `sz_max` vo percentách šírky snímky).
8. **Výstupové obrázky** – pôvodný snímok a maska s nakreslenými kľúčovými bodmi (`drawKeypoints`) a obrysom okna; pre ladenie existuje voliteľné okno s posuvníkmi (`tuning_mode`) meniacimi HSV a okno súčasne.

Uzol `detect_ball` z najväčšieho nájdeneho kľúčového bodu naplní správu `geometry_msgs/msg/Point`: `x`, `y` sú **normalizované súradnice** relatívne voči stredu obrazu (približne v rozsahu $[-1, 1]$), `z` nesie relatívnu veľkosť blobu (použije sa ako hrubý indikátor vzdialenosti).

Uzol `follow_ball` odoberá `/detected_ball` a v časovači publikuje `/cmd_vel`. Horizontálna odchýlka `x` cez exponenciálny filter riadi `angular.z` (proporcionálne sledovanie stredu); pri dostatočne “malej” lopte v obraze (`z` pod prahom `max_size_thresh`)

sa pridá `linear.x` (jazda vpred, kým je objekt ďaleko). Pri strate signálu dlhšom ako `rcv_timeout_secs` robot otáča hľadanie (`search_angular_speed`).

Voliteľný uzol `detect_ball_3d` z odhadnutej uhlovej veľkosti a zadaného polomeru lopty dopočíta približnú vzdialenosť a publikuje bod v priestore a `visualization_msgs/msg/Marker` (sféra) pre RViz.

V README autora je uvedené, že **červená** farba je v HSV problematická kvôli obtočeniu odtieňa okolo $0^\circ / 360^\circ$; robustná detekcia červenej by vyžadovala úpravu dvoch HSV intervalov alebo inú reprezentáciu farby.

Konkrétne snímky z **vlastnej implementácie** sledovania (debug okno a priebeh veličín) sú uvedené v podsekcii nižšie na obrázkoch 4.12 a 4.13.

Pred alternatívnou vetvou s ESP32 dáva zmysel **zhrnúť uzly** jednou schémou: nasledujúca sekcia je preto prehľadná (bloková) doplnok k podrobnému rozboru vyššie.

Vlastná implementácia sledovania lopty s adaptívnym PID regulátorom

V tejto práci bolo sledovanie lopty implementované vlastným spôsobom ako ROS 2 balík `waverower`, pričom oproti referenčnému príkladu (`ball_tracker`) boli zavedené viaceré vylepšenia: stavový automat s fázami hľadania a sledovania, akciový server (`FollowBall`) s podporou stromu správania (Behavior Tree), a predovšetkým **adaptívne riadenie uhlovej rýchlosti** odvodzujúce zosilnenie regulátora od zdanlivej veľkosti lopty v obraze.

Motivácia: obmedzenia konštantného zosilnenia. Klasický proporcionálny regulátor s konštantným zosilnením K_p používa horizontálnu polohovú chybu e (normalizovanú odchýlku stredu lopty od stredu obrazu) priamo ako vstup:

$$\omega_z = K_p \cdot e \quad (4.1)$$

Tento prístup má praktické nedostatky: pri **veľkej vzdialenosti** (malý obraz lopty) je reakcia príliš pomalá a robot nestíha sledovať pohybujúci sa objekt, zatiaľ čo pri **malej vzdialenosti** (veľký obraz) môže byť reakcia agresívna a spôsobovať oscilácie alebo nežiaducu reverziu kolies.

Teoretický základ: interakčná matica IBVS. Riešenie vychádza z teórie obrazového sérvoriadenia (*Image-Based Visual Servoing*, IBVS). Interakčná matica (Jacobian obrazu) popisuje vzťah medzi pohybom bodu v obrazovej rovine a rýchlosťami kamery [7]. Pre horizontálnu súradnicu bodu x v normalizovaných súradniciach platí:

$$\dot{x} = -\frac{v_x}{Z} + \frac{x v_z}{Z} + xy \omega_x - (1 + x^2) \omega_z + y \omega_y \quad (4.2)$$

kde Z je hĺbka objektu od kamery. Pre čistú rotáciu okolo zvislej osi ω_z dominuje člen $-(1 + x^2) \omega_z$. Keďže zdanlivý polomer lopty r_{px} v obraze je nepriamo úmerný vzdialenosti:

$$r_{px} \propto \frac{1}{Z} \quad (4.3)$$

vyplýva, že pri väčšej vzdialenosti (menší r_{px}) treba väčší zosilňovací koeficient, aby sa dosiahol rovnaký uhlový rýchlosti sledovania. Inými slovami, **adaptívny zisk by mal rásť so zdanlivou veľkosťou lopty**.

Implementácia adaptívneho ziskového koeficientu. Adaptívne zosilnenie je realizované škálovaním proporcionálnej zložky K_p podľa pomeru aktuálneho polomeru r_{px} ku referenčnému polomeru r_{ref} (polomer pri nominálnej vzdialenosti):

$$K_{eff} = K_p \cdot \text{clip}\left(\frac{r_{px}}{r_{ref}}, s_{\min}, s_{\max}\right) \quad (4.4)$$

kde $s_{\min}$ a $s_{\max}$ sú dolná a horná hranica škálovania (ochrana pred extrémami). Výsledný PID regulátor pre uhlovú rýchlosť má tvar:

$$\omega_z = K_{eff} \cdot e + K_i \int e \, dt + K_d \frac{de}{dt} \quad (4.5)$$

kde $e = \frac{c_x - w/2}{w/2}$ je normalizovaná horizontálna odchýlka (c_x – x-súradnica stredu lopty, w – šírka obrazu v pixeloch). V implementácii sú použité parametre $K_p = 0,5$, $K_i = 0,02$, $K_d = 0,05$, $r_{ref} = 30$ px, $s_{\min} = 0,2$, $s_{\max} = 2,0$.

Správanie pri rôznych vzdialenostiach. Tabuľka 4.7 uvádza efektívny zisk pre rôzne zdanlivé polomery lopty:

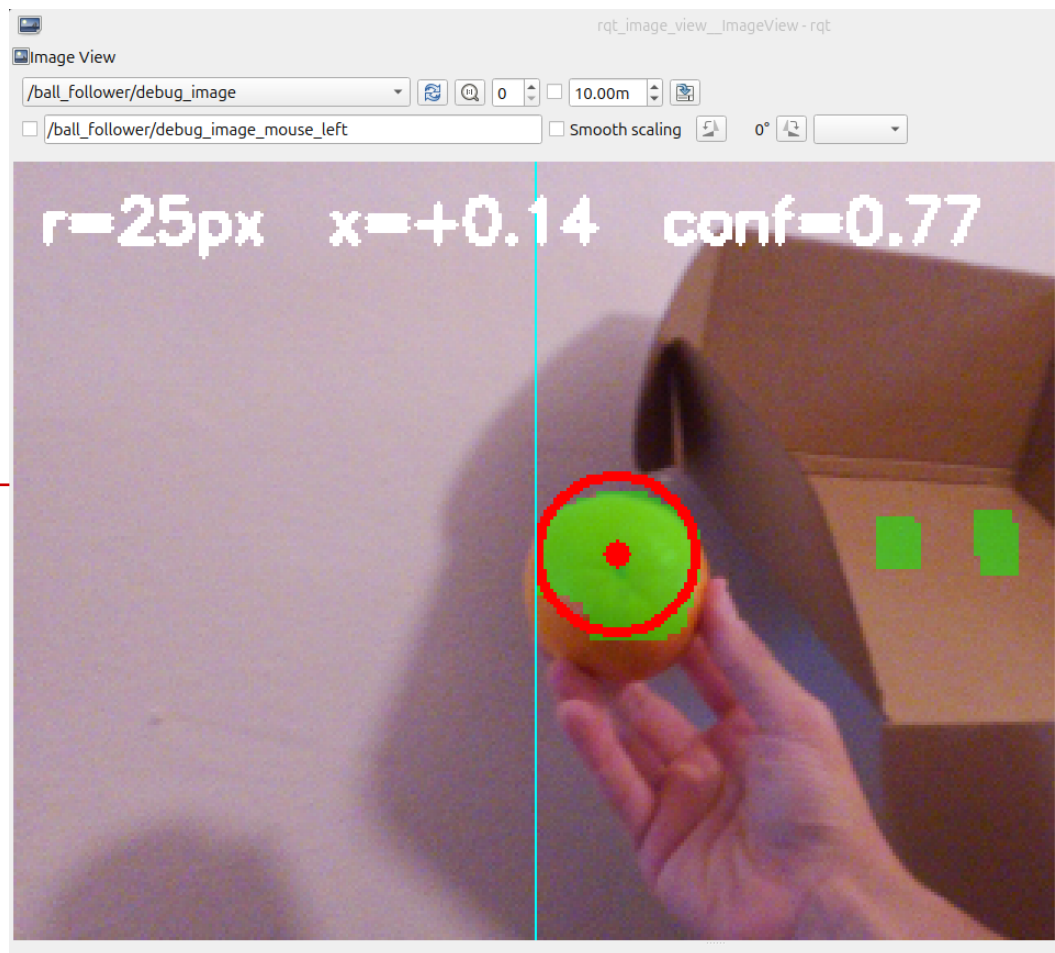
Tabuľka 4.7: Adaptívny efektívny zisk PID regulátora v závislosti od zdanlivého polomeru lopty

r_{px} [px]	Vzdialenosť	Škálovací faktor	K_{eff}
10	veľmi ďaleko	$0,33 \rightarrow \text{clip } 0,20$	0,10
30	stredná (ref.)	1,00	0,50
50	blízko	1,67	0,83
60	veľmi blízko	2,00 (cap)	1,00

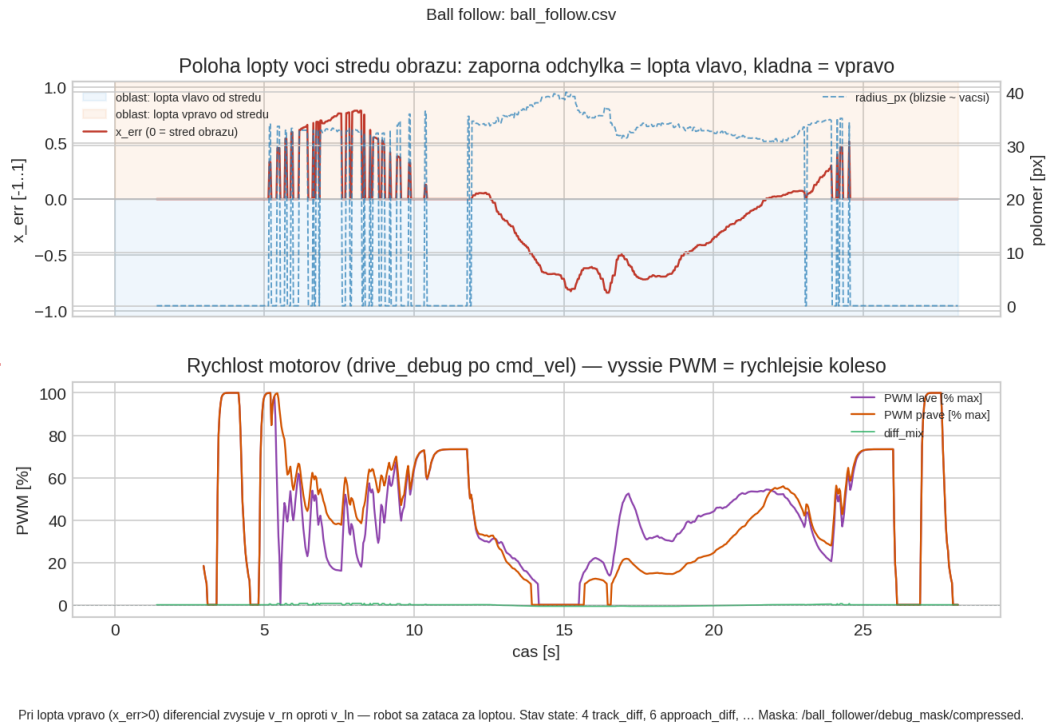
Stavový automat a Behavior Tree. Sledovanie je organizované do dvoch fáz riadených stromom správania (*Behavior Tree*) implementovaným pomocou knižnice `py_trees`:

- **FindBall** – robot vykonáva smerové výkyvy (*bursty*) v smere kde naposledy videl loptu; fáza končí úspechom (**SUCCESS**) pri prvej platnej detekcii.
- **FollowBall** – robot sleduje loptu s PID reguláciou; fáza končí úspechom pri dosiahnutí dostatočnej blízkosti ($r_{px} \geq r_{stop}$) alebo zlyhaním (**FAILURE**) pri strate lopty na viac ako 1,5 s.
- **Celebrate** – krátke otočenie na mieste ako vizuálna spätná väzba po dosiahnutí cieľa.

Pri zlyhaní fázy **FollowBall** (strata lopty) **Sequence** zlyhá a **Repeat** dekorátor reštartuje strom od fázy **FindBall**, pričom si robot pamätá stranu, kde loptu videl naposledy.



Obr. 4.12: Debug výstup sledovania objektu v `rqt_image_view` (téma `/ball_follower/debug_image`): maska, odhad polomeru r , odchýlka x a spoľahlivosť detekcie



Obr. 4.13: Priebeh regulačných veličín pri sledovaní objektu (uhlová a lineárna rýchlosť v čase)

4.5 Návrh uzlov systému pre ROS2

Navrhovaný systém pre riadenie robota Wave Rover v ROS 2 nie je jediný monolitický program, ale **súbor uzlov** prepojených témami. Tabuľka 4.8 zhrňuje *skutočne použité* bloky a kde sa nachádzajú v kóde; širšie dôvody modularizácie sú rozvedené v texte kapitoly. Grafické prepojenia z reálneho behu sú na snímkach `rqt_graph` vyššie (obr. 4.3, 4.9, 4.11) — namiesto abstraktnej schémy, ktorá by nezodpovedala presným názvom uzlov v kóde.

Uzly a ich funkcie v systéme

V tabuľke 4.8 je v tretom stĺpci uvedený len stručný účel bloku, nie samostatné zdôvodnenie jeho oddelenia od ostatných častí systému.

Tabuľka 4.8: Prehľad hlavných logických blokov a ich implementácie v projekte

Blok	Implementácia (kde)	Účel
LiDAR	balík <code>ldlidar_ros2</code>	Publikuje <code>/scan</code> pre wander a SLAM; v simulácii rovnaký typ <code>LaserScan</code> využíva aj Nav2 vďaka štandardnému rozhraniu a hotovým algoritmom bez vlastného drivera.
Kamera	<code>camera_ros</code> , web UI	Obraz pre operátora a pre optický tok; komprimovaný tok znižuje záťaž CPU a siete pri teleprezencii.
IMU	firmware (Nano), <code>imu_serial_fusion_bridge.py</code>	Téma <code>/imu</code> slúži ako podklad pre PID na motore a meranie otočenia pri wanderi; údaje po USB z Nano obchádzajú konflikt I ² C s HATom (sek. 4.6).
Pohon	<code>MotorHatNode</code> (<code>motor_hat.cpp</code>)	Jediný výstup na PWM; režimy <code>manual</code> a <code>auto</code> oddelia teleop od autonómie.
Teleop	klávesnica, <code>web_ui/</code> , <code>rosbridge</code>	Spoločná téma <code>/teleop_cmd_vel</code> ; webové UI umožní ovládanie z mobilu bez inštalácie aplikácie.
Korekcia dráhy	<code>optical_flow_node</code>	Voliteľná úprava <code>/teleop_cmd_vel_corrected</code> ; vizuálnu vetvu možno vypnúť bez zásahu do motorového uzla.
Bludenie	<code>lidar_wander.py</code>	Jednoduché reaktívne správanie bez mapy, demonštrácia autonómie s LD19.
Prepínanie režimu	<code>robot_mode_switch.py</code>	Služby a vzdialená zmena parametrov, prepínanie bez reštartu uzlov (sek. 4.4).
SLAM	<code>slam_toolbox</code> [6]	2D mapa z laseru; pre základnú mapu nie je nutná kamera.
Globálna navigácia	Nav2 (iba simulácia)	Plánovanie v známej mape v Gazebo (iná logika než wander, sek. 4.4); na fyzickom robote v rámci práce neuvedené.
Sledovanie objektu	vlastná vetva, inšpirácia <code>ball_tracker</code>	Počítačové videnie a adaptívny PID, oddelené od bezpečnostnej navigácie (Nav2, wander).
Prevádzka	voliteľné <code>systemd</code> unit súbory	Automatický štart po boote a reštart po páde; vhodné pre dlhé merania v teréne.

4.6 Implementácia uzlov a komunikácia s UGV Base General

Poznámka: Táto sekcia popisuje *alternatívnu* komunikáciu s firmwareom UGV (ESP32, HTTP) a referenčné C++ uzly v `waverower/src/esp32`. **Hlavná nasadená vetva** na Wave Roveri je v balíku `waverower` s Motor Driver HAT a štandardnými názvami ROS 2 tém (bez umelého prefixu) – sek. 4.4 a nižšie zhrnutie tém.

Nižšie sú HTTP požiadavky firmwaru *UGV Base General* na ESP32 a referenčné C++ uzly v `waverower/src/esp32`. **Samostatný uzol s názvom „ESP32 Bridge“ v tomto repozitári nemusí existovať** – mostenie je rozdelené medzi uvedené spustiteľné súbory a prípadné príklady Waveshare. Oficiálna dokumentácia a firmware sú v repozitári `ugv_base_general` [36]; lokálna kópia je v `ugv_base_general/`, súbor `General_Driver/json_cmd.h` enumeruje konštanty príkazov (`CMD_*`) a ukážkové JSON reťazce.

Úloha Raspberry Pi a ESP32 (master–slave)

V konfigurácii s doskou *General Driver* (ESP32) plní **Raspberry Pi úlohu horného počítača (master)**: beží na ňom ROS 2, senzorové uzly (napr. LiDAR cez USB), plánovanie a uzol mosta. **ESP32 je dolný počítač (slave)**: priamo riadi motory (PID podľa dokumentácie výrobcu), číta IMU pripojené k doske a vystavuje rozhranie pre príkazy. Komunikácia neprebíha cez ROS 2 DDS priamo na mikrokontroléri, ale **HTTP GET** (alebo podľa výrobcu aj sériový UART/USB s rovnakým JSON) — Raspberry Pi teda posiela textové príkazy a pri pollingu IMU číta JSON odpovede. Tým sa oddelí reálny čas riadenia na ESP32 od linuxového stacku na RPi.

Referenčné C++ uzly v `waverower/src/esp32`

V balíku `waverower` sú (mimo šablóny `ugv_base_general`) uložené **dve jednoduché referenčné implementácie** mosta medzi ROS 2 a rovnakým HTTP API (`/js?json=...`), vhodné na ladenie podvozku s firmwareom UGV bez Motor Driver HAT na RPi:

- **`motor_http_teleop_node.cpp`** – uzol `motor_http_teleop` číta ovládanie z **terminálu** (raw režim `stdin`): šípky, WASD, úprava rýchlosti (+/-, číslce 1–9), nezávislý *trim* ľavého a pravého kanála (klávesy `u/i`, `o/p`). Periodický časovač (typicky 10–100 ms) vyhodnocuje stlačené smery, rieši súčasné stlačenia (napr. `vpred+vpravo` = oblúk) a konflikty dopredu/dozadu. Výstup je vždy JSON `{"T":1,"L":L,"R":R}` s $L, R \in [-1, 1]$, odoslaný cez **libcurl** ako HTTP GET na `http://<esp32_ip>/js`. Parametre: `esp32_ip`, `speed`, `left_trim`, `right_trim`, `key_release_timeout_ms` (aby držanie klávesa nespôsobilo prerušenie po `key-repeat`). Pri ukončení uzla sa pošle zastavenie motorov (0,0).
- **`motor_driver_node.cpp`** – uzol `motor_driver` odoberá `std_msgs/msg/Float64MultiArray` na téme `/motor_cmd` (očakáva aspoň dve zložky: ľavé a pravé koleso v rozsahu $-1 \dots 1$), každá zmena správy sa okamžite mapuje na rovnaký HTTP príkaz `T:1`. Slúži na pripojenie vlastného plánovača alebo skriptu, ktorý už vypočíta normované rýchlosti kolies.

Obe varianty zdieľajú rovnaké URL-kódovanie JSON reťazca ako v podsekciiach nižšie; v CMakeLists môžu byť voliteľné ciele podľa toho, či sa má zostavovať len HAT vetva alebo

aj ESP32 pomocné nástroje. **TODO (schéma):** blokový diagram RPi → HTTP → ESP32 → H-mósty → motory.

HTTP volania voči ESP32 (UGV Base General)

Komunikácia medzi ROS 2 na Raspberry Pi a doskou s ESP32 prebieha podľa nižšie uvedeného protokolu (GET s parametrom `json`). Referenčné uzly `motor_http_teleop` a `motor_driver` používajú rovnaké volania cez `libcurl`; ROS 2 témy v hlavnej vetve sú v zozname nižšie, nie pod umelým prefixom.

Formát požiadaviek

UGV Base General poskytuje HTTP server na porte 80 (v lokálnej sieti Wi-Fi), ktorý prijíma JSON príkazy cez GET požiadavky s query parametrom `json`. Formát URL je:

`http://<ESP32_IP>/js?json=<URL_ENCODED_JSON>`

Klientský kód (napr. v referenčných uzloch vyššie) používa knižnicu `libcurl` na odoslanie HTTP požiadaviek. Každý príkaz je JSON objekt s polom `T` (typ príkazu) a ďalšími parametrami podľa typu príkazu; zoznam typov je v `json_cmd.h` [36].

Príklady požiadaviek (reťazec `json` musí byť URL-kódovaný).

- **Rýchlosť kolies (normovaná)** — `{"T":1,"L":0.3,"R":0.3}`: jazda vpred obe strany 0,3 (rozsah typicky $-1 \dots 1$). Po zakódovaní má požiadavka tvar: `/js?json=%7B%22T%22%3A1%2C%22L%22%3A0.3%2C%22R%22%3A0.3%7D`.
- **PWM priamo** — `{"T":11,"L":164,"R":164}`: hodnoty PWM ± 255 (stred 164 = stoj), podľa `CMD_PWM_INPUT` vo firmware.
- **ROS štýl** (v, ω) — firmware pozná aj `{"T":13,"X":0.1,"Z":0.3}` (m/s a rad/s), ak je to podporované konkrétnym podvozkom s enkodermi (v `json_cmd.h` je poznámka *Not for the products without encoders*).
- **Čítanie IMU** — `{"T":126}`: odpoveď s typom spätného kanála `T:1002` a akceleráciou/gyro (podrobnosti v protokole výrobcu).

Testovanie mimo ROS 2 je možné napr. príkazom `curl` z Raspberry Pi (nahradením `<ESP32_IP>` reálnou adresou):

```
curl "http://<ESP32_IP>/js?json=%7B%22T%22%3A126%7D"
```

Hypotetický uzol typu „bridge“ by z týchto HTTP volaní vedel skladať ROS 2 správy; **v hlavnej vetve tohto repozitára** sa IMU na zostave s Motor HAT neberie z ESP32, ale z USB podľa sekcie 4.6.

ROS 2 témy v projekte waverower (skutočné názvy)

V texte práce **sa nepoužívajú** umelo zvolené prefixy typu `/bpc_prp_robot/...` (šlo o starú šablónu z iného dokumentu). Typické témy pri Motor Driver HAT zostave sú v tabuľke 4.9; tretí stĺpec stručne uvádza účel témy (kto ju typicky plní je súčasťou popisu).

Tabuľka 4.9: Typické ROS 2 témy v hlavnej vetve (Motor Driver HAT na RPi)

Téma	Typ správy	Účel
/scan	sensor_msgs/msg/LaserScan	Laser LD19 cez ldlidar_ros2 pre SLAM a wander; Nav2 v simulácii používa rovnaký typ dát.
/imu	sensor_msgs/msg/Imu	imu_serial_fusion_bridge.py (Nano): podklad pre PID, wander a EKF podľa launch.
/teleop_cmd_vel	geometry_msgs/msg/Twist	Manuálny vstup z klávesnice alebo web UI.
/teleop_cmd_vel_corrected	geometry_msgs/msg/Twist	Optický tok upravuje teleop pred vstupom do motorového uzla.
/cmd_vel	geometry_msgs/msg/Twist	Autonómny príkaz z wanderu alebo CV; Nav2 len pri simulačnom behu.
/camera/image_raw	surový obraz	camera_ros: vizualizácia a ladenie.
/camera/.../compressed	komprimovaný obraz	Teleprezencia a optický tok s nižšou záťažou.
/motor_cmd	std_msgs/msg/Float64MultiArray	Len referenčný ESP32 uzol motor_driver.

JSON motorového príkazu {"T":1,"L":...,"R":...} zostáva rovnaký na úrovni HTTP; mapovanie z ROS sa v referenčných uzloch robí priamo z Twist alebo Float64MultiArray, nie z vlastných tém cmd_motor_left / cmd_motor_right. Šablónové uzly s prefixom /bpc_prp_robot/ z cudzích dokumentov sa v waverower nepoužívajú – skutočné uzly a témy sú v sekcii 4.4.

MPU6050 na Arduino Nano a USB (vetva s Motor Driver HAT na RPi)

Pri zostave, kde Raspberry Pi priamo ovláda motory cez Waveshare Motor Driver HAT po I²C (PCA9685, typicky adresa 0x40), bol čip MPU-6050 (adresa 0x68) pôvodne uvažovaný na **tej istej I²C zbernici** ako HAT. V praxi sa ukázalo, že **súbežný prístup** – súčasné alebo striedavé čítanie a zápis na zbernici medzi ovládaním PWM motorov a čítaním IMU – vedie k problémom: nestabilné správanie, straty transakcií alebo kolízie pri zdieľaní zbernice. Pokus o koordináciu prístupov na strane Linuxu (**mutexy** medzi procesmi/uzlami) problém *nedostatočne vyriešil*; zbernica a načasovanie ovládača HATu zostali kritickým miestom. Praktické riešenie preto bolo **oddeliť IMU na vlastný mikrokontrolér**: Arduino Nano má výhradne svoju lokálnu I²C medzi sebou a MPU-6050, zatiaľ čo RPi komunikuje s HATom bez súperenia o ten istý fyzický I²C k IMU.

Vlastný firmware v súbore waverower/firmware/mpu6050_serial/mpu6050_serial.ino beží na **Arduino Nano**. MPU-6050 je pripojený len k Nano cez I²C (Wire, typicky

100 kHz, adresa 0x68), s inicializáciou registrov (prebudenie, DLPF, rozsahy gyroskopu a akcelerometra, vzorkovanie). Po štarte sa vykoná **kalibrácia gyroskopu** pri nehybnom senzore.

V hlavnej slučke sa čítajú surové registre, prepočítajú sa na fyzikálne jednotky, použije sa komplementárny filter pre roll/pitch a integrácia gyroskopu pre yaw (bez magnetometra zostáva očakávaný drift yaw). Výstup je **jedna textová CSV línia na riadok** cez USB sériový port (115200 Bd).

Na Raspberry Pi uzol `imu_serial_fusion_bridge.py` číta tento stream, voliteľne dopĺňa ďalšiu úpravu a publikuje `sensor_msgs/msg/Imu` na tému `/imu`. Celá vetva je zámerne jednoduchá na ladenie (Arduino IDE, sériový monitor) a nezávislá od ROS buildu firmwaru HATu.

Kapitola 5

Záver

Cieľom práce bolo navrhnuť a implementovať riadiaci softvér pre mobilnú platformu Wave Rover s využitím ROS 2 na Raspberry Pi 5, s dôrazom na zrozumiteľnú architektúru, merateľné správanie v praxi a zladenie so štandardnými nástrojmi komunity (RViz, `slam_toolbox`, Nav2 v simulácii, `rqt`).

V práci bol popísaný hardvér podvozku a senzorov, zvolený softvérový stack ROS 2 (distribúcia Jazzy) a najmä vlastný balík `waverover`, ktorý združuje ovládanie motora cez Motor Driver HAT, spracovanie LiDARu LD19, most IMU z Arduino Nano po USB, voliteľnú korekciu jazdy (IMU PID, optický tok), autonómne bludenie, prepínanie režimov za behu, webovú teleprezenciu, mapovanie (SLAM) a integráciu Nav2 **v simulácii**. Overenie plnej globálnej navigácie je súčasťou simulačného workspace s Gazebo Sim a modelom robota v URDF/Xacro; na fyzickom podvozku zostáva autonómia cez bludenie a teleop so SLAM. Ako doplnok bola rozvinutá vetva sledovania farebného objektu s adaptívnym PID a stromom správania. Text zámerne uvádza nielen *čo* je implementované, ale aj *prečo* boli zvolené jednotlivé formy rozdelenia uzlov, tém a launch súborov.

V praxi zostávajú obmedzenia dané platformou: chýbajúce enkodéry kolies sťažujú presnú odometriu v rovine; riešenie cez integráciu `/cmd_vel` a IMU je len približné. Yaw z MPU-6050 bez magnetometra driftuje a optický tok závisí od osvetlenia a textúr prostredia. Tieto body však nepopierajú zmysel riešenia — ukazujú, kde má zmysel ďalšie vylepšenie hardvéru alebo algoritmov.

Do budúca dáva zmysel zlepšiť odometriu — napríklad enkodérmi na kolesách alebo vizuálnou či lidarovou zložkou —, doladenie parametrov regulátorov a ďalšie scenáre navigácie či úlohy počítačového videnia na tom istom ROS 2 rozhraní, ktoré táto práca ustálila.

Literatúra

- [1] ADAMS, V. H. *Complementary Filters* online. 2024. Dostupné z: https://vanhunteradams.com/Pico/ReactionWheel/Complementary_Filters.html. [cit. 2024-01-25].
- [2] ANAND, A. Accelerometer Working Principle. *Intelligent Detection and Sensing Systems*. 1. vyd., 2023, zv. 1, č. 1, s. 4.
- [3] ARLUNAN a CONTRIBUTORS. *Raspberry Pi Camera Installation on Raspberry Pi 5 / Ubuntu 24.04 Noble / ROS 2 Jazzy* online. 2026. Dostupné z: <https://github.com/ARLunan/Raspberry-Pi-Camera-ROS/blob/main/Documents/RPCamera-Installation-RP5.md>. [cit. 2026-02-21]. Dokumentácia inštalácie kamery pre RPi5, Ubuntu 24.04 a ROS 2 Jazzy.
- [4] ÅSTRÖM, K. J. a MURRAY, R. M. *Feedback Systems: An Introduction for Scientists and Engineers*. 1. vyd. Princeton University Press, 2008. ISBN 978-0-691-13576-2.
- [5] BEHROOZPOUR, B.; POULAIN, P.; GAMAL, A. E.; BOSER, B. E. a JALALI, B. LiDAR System Architectures and Circuits. *Proceedings of the IEEE*. 1. vyd., 2017, zv. 105, č. 10, s. 1960–1971.
- [6] CADENA, C.; CARLONE, L.; CARRILLO, H.; LATIF, Y.; SCARAMUZZA, D. et al. Past, Present, and Future of Simultaneous Localization and Mapping: Toward the Robust-Perception Age. *IEEE Transactions on Robotics*. 1. vyd., 2016, zv. 32, č. 6, s. 1309–1332.
- [7] CHAUMETTE, F. a HUTCHINSON, S. Visual servo control, part I: Basic approaches. *IEEE Robotics and Automation Magazine*. 1. vyd., 2006, zv. 13, č. 4, s. 82–90. Prehľad obrazového sérvoriadenia (IBVS).
- [8] ELECTRONIC CATS. *MPU6050* online. 2024. Dostupné z: <https://github.com/ElectronicCats/mpu6050>. [cit. 2026-03-28]. Popis čipu MPU6050, rozsahy gyroskopu a akcelerometra, I2C.
- [9] FEDOROV, D. S.; IVOILOV, A. Y.; ZHMUD, V. A. a TRUBIN, V. G. Using of measuring system MPU6050 for the determination of the angular velocities and linear accelerations. *Automatics & Software Engineering*. 1. vyd., 2015, zv. 11, č. 1, s. 75–80.
- [10] GUNTURK, B. K.; GLOTZBACH, J.; ALTUNBASAK, Y.; SCHAFER, R. W. a MERSEREAU, R. M. Demosaicking: Color Filter Array Interpolation. *IEEE Signal Processing Magazine*. 1. vyd., 2005, zv. 22, č. 1, s. 44–54.

- [11] MACENSKI, S.; FOOTE, T.; GERKEY, B.; LALANCETTE, C. a WOODALL, W. Robot Operating System 2: Design, architecture, and uses in the wild. *Science Robotics*. 1. vyd., 2022, zv. 7, č. 66, s. eabm6074. Dostupné z: <https://www.science.org/doi/abs/10.1126/scirobotics.abm6074>.
- [12] MANKAR, J.; DARODE, C.; TRIVEDI, K.; KANOJE, M. a SHAHARE, P. Review of I2C protocol. *International Journal of Research in Advent Technology*. 1. vyd. Citeseer, 2014, zv. 2, č. 1.
- [13] MICRO-ROS PROJECT. *Micro-ROS – ROS 2 for microcontrollers* online. 2024. Dostupné z: <https://micro.ros.org/>. [cit. 2025-02-22]. Oficiálna stránka projektu micro-ROS.
- [14] MICRO-ROS PROJECT. *Micro_ros_setup – Building and flashing micro-ROS* online. 2024. Dostupné z: https://github.com/micro-ROS/micro_ros_setup. [cit. 2025-02-22]. Nástroje na zostavenie a nahratie micro-ROS firmware.
- [15] NEWANS, J. *Ball_tracker* online. 2023. Dostupné z: https://github.com/joshnewans/ball_tracker. [cit. 2026-03-28]. ROS 2 Python balík: HSV segmentácia, SimpleBlobDetector, sledovanie lopty.
- [16] OBJECT MANAGEMENT GROUP / DDS FOUNDATION. *What is DDS?* online. 2024. Dostupné z: <https://www.dds-foundation.org/what-is-dds-3/>. [cit. 2026-03-28]. Štandard middleware pod ROS 2.
- [17] ONDERKA, D. *Mobilní kamera realizovaná prostředky ROS2*. Brno, 2024. Bakalářská práce. Vysoké učení technické v Brně, Fakulta informačních technologií. Dostupné z: <https://www.vut.cz/studenti/zav-prace/detail/155048>. [cit. 2026-04-08]. Vedoucí práce doc. Ing. Vladimír Janoušek, Ph.D. Adept AWR 4WD, teleprezence, Nav2, slam_toolbox.
- [18] OPEN NAVIGATION LLC ET AL.. *Navigation2* online. 2025. Dostupné z: <https://navigation.ros.org/>. [cit. 2026-03-28]. Oficiálna dokumentácia stacku Nav2 pre ROS 2.
- [19] OPEN ROBOTICS. *About discovery* online. 2025. Dostupné z: <https://docs.ros.org/en/jazzy/Concepts/Basic/About-Discovery.html>. [cit. 2026-03-28]. Vyhľadávanie uzlov v DDS doméne.
- [20] OPEN ROBOTICS. *About Quality of Service settings* online. 2025. Dostupné z: <https://docs.ros.org/en/jazzy/Concepts/Intermediate/About-Quality-of-Service-Settings.html>. [cit. 2026-03-28]. History, reliability, durability a ďalšie QoS politiky.
- [21] OPEN ROBOTICS. *Common_interfaces: standard message definitions for ROS 2* online. 2025. Dostupné z: https://github.com/ros2/common_interfaces. [cit. 2026-04-08]. Geometry_msgs, sensor_msgs, nav_msgs a ďalšie.
- [22] OPEN ROBOTICS. *ROS 2 Documentation* online. 2025. Dostupné z: <https://docs.ros.org/en/jazzy/index.html>. [cit. 2026-04-08]. Distribúcia Jazzy; tutoriály a koncepty.

- [23] OPEN ROBOTICS. *Understanding ROS 2 topics* online. 2025. Dostupné z: <https://docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Understanding-ROS-2-Topics/Understanding-ROS2-Topics.html>. [cit. 2026-03-28]. Témy, publisher/subscriber, typy správ.
- [24] OPENCV. *Optical Flow* online. 2024. Dostupné z: https://docs.opencv.org/3.4/d4/dee/tutorial_optical_flow.html. [cit. 2026-03-28]. Tutoriál: riedky tok Lucas–Kanade a hustý tok Farnebäck.
- [25] RASPBERRY PI FOUNDATION. *Processors* online. Raspberry Pi Foundation, 2024. Dostupné z: <https://www.raspberrypi.com/documentation/computers/processors.html#bcm2712>. [cit. 2024-11-02].
- [26] RASPBERRY PI LTD. *Documentation: Raspberry Pi computers and microcontrollers* online. 2024. Dostupné z: <https://www.raspberrypi.com/documentation/>. [cit. 2026-04-08]. Oficiálna dokumentácia hardvéru a GPIO.
- [27] RASPBERRY PI LTD. *Camera Module* online. 2025. Dostupné z: <https://www.raspberrypi.com/documentation/accessories/camera.html>. [cit. 2026-03-28]. Hardvér CMOS senzora pre Raspberry Pi Camera.
- [28] RAUCH, C. *Camera_ros: ROS 2 driver for libcamera* online. 2024. Dostupné z: https://github.com/christianrauch/camera_ros. [cit. 2025-02-21]. Balík pre publikovanie obrazu z kamery na ROS 2 témy.
- [29] ROBOT WEB TOOLS. *Rosbridge_suite* online. 2024. Dostupné z: https://github.com/RobotWebTools/rosbridge_suite. [cit. 2026-03-28]. WebSocket most medzi prehliadačom a ROS 2 (rosbridge).
- [30] ROBOT WEB TOOLS. *Roslibjs* online. 2024. Dostupné z: <https://github.com/RobotWebTools/roslibjs>. [cit. 2026-03-28]. JavaScript klientská knižnica pre ROS cez rosbridge.
- [31] SIEGWART, R.; NOURBAKHS, I. R. a SCARAMUZZA, D. *Introduction to Autonomous Mobile Robots*. 2nd. MIT Press, 2011. ISBN 978-0-262-01535-4.
- [32] SIM, G. *Building a Robot* online. 2024. Dostupné z: https://gazebo-sim.org/docs/latest/building_robot/. [cit. 2024-01-25].
- [33] SKOLEK, T. *Balancující dvoukolový robot*. Brno, 2025. Bakalářská práce. Vysoké učení technické v Brně, Fakulta informačních technologií. Vedoucí práce doc. Ing. Vladimír Janoušek, Ph.D. ROS2 (Humble), RPi4B, PID, IMU, enkodéry, Bluetooth, autonomní pohyb.
- [34] WAVESHARE. *General Driver for Robots* online. Waveshare, 2024. Dostupné z: https://www.waveshare.com/wiki/General_Driver_for_Robots. [cit. 2024-01-25].
- [35] WAVESHARE. *WAVE ROVER* online. Waveshare, 2024. Dostupné z: https://www.waveshare.com/wiki/WAVE_ROVER. [cit. 2024-01-25].
- [36] WAVESHARE / EFFECTSMACHINE. *Ugv_base_general* online. 2024. Dostupné z: https://github.com/effectsmachine/ugv_base_general. [cit. 2026-03-28]. Firmware General Driver pre ESP32; JSON príkazy a README.